Retro Computers on the BeagleV-Fire

Six machines in one FPGA bitstream, and a web page as the whole front end

Technical paper, state of 2 November 2026. Project: BeagleV-Fire-Retro (github.com/costarc/BeagleV-Fire-Retro). Gateware 4328, package 0.14.5.

Abstract

The BeagleV-Fire is a Linux board built around a Microchip PolarFire SoC: four 64-bit RISC-V application cores and about 23K logic elements of FPGA fabric on one chip. It has no video connector for the fabric, no keyboard port, no audio output and no storage the fabric can reach. We run six 1980s computers in that fabric anyway: the Prológica CP-300 (a Brazilian TRS-80 Model III compatible), the CP-400 (a TRS-80 Color Computer 2 compatible), an MSX1, a ZX Spectrum 48K, the CCE MC-1000 and, with a processor that is not in the fabric at all, the Matra Alice (a Tandy MC-10 clone). One bitstream holds all of them; the Linux side selects the machine by writing one register, with no reflash and no reboot.

Everything a user touches (screen, keyboard, speaker, floppy drives, cassette recorder, hard disk) is provided by **a web page**, served by a small Python daemon on the board's Linux, which talks to the fabric through **one 1 MB memory-mapped register window**.

This paper documents the design and, above all, *why* each decision went the way it did. It is organised around the **architecture variants** we had to design: one CPU shared by four machines, one block-RAM pool shared by all of them, ROMs in DDR behind a cache, a 6809 machine with its own CPU, and a machine whose CPU runs on the host. It explains in detail why the Alice uses a soft CPU instead of an FPGA core, why the MSX video memory has to be on the chip (and what the abandoned MSX2 taught us about it), how three different floppy controllers share one WD1793 and one host-side sector service, and what went wrong on the way. The retro computers are the example; the subject is the interface problem that appears whenever a hardware block sits next to a general-purpose CPU: moving a video stream, event streams, bulk storage and control state across the processor/fabric boundary reliably, cheaply, and in a way a non-specialist can use from a browser.

Contents

1. The problem and the constraints 3
- 1.1 The board 3
- 1.2 Requirements we set ourselves 3
- 1.3 Where this started 3

2. Architecture overview 4

3. Architecture variants: what we built and what we threw away 5

4. Key decisions and the reasons 7

5. The platform: what the BeagleV-Fire gives you 9

6. The FPGA/CPU interface 9
- 6.1 The address map 9
- 6.2 Crossing from the 50 MHz bus to the 42 MHz core 10
- 6.3 Control, the start sequence and the three hold states 10
- 6.4 The read-latency convention 10

7. The machines 11
- 7.1 One Z80 for four machines 11
- 7.2 One block-RAM pool 12
- 7.3 CP-300 (machine 0) 12
- 7.4 CP-400 / CoCo 2 (machine 1): a 6809 machine 13
- 7.5 MSX1 (machine 2) 14
- 7.6 ZX Spectrum 48K (machine 3) 15
- 7.7 MC-1000 (machine 4) 15
- 7.8 Matra Alice / Tandy MC-10 (machine 5): the CPU on the host 16

8. Why the Alice uses a soft CPU instead of an FPGA core 16

8.1 What the machine needs....16
8.2 The options....17
8.3 What crosses the boundary, and what does not....17
8.4 Why this is better than it looks....18
8.5 What is not known....18
9. Why the video memory must be on the chip, and what the MSX2 taught us....19
9.1 The rule: the CPU may wait, the video may not....19
9.2 Where each memory of the MSX1 lives....20
9.3 The MSX2: what went wrong with 128 KB....20
9.4 Is it still true?....20
10. Disks, tape and sound....21
10.1 One WD1793, three front ends....21
10.2 The request/response protocol....21
10.3 Building a front end for a ROM you cannot change: the MSX disk interface....22
10.4 The CP-400's hard disk: two puzzles....22
10.5 Tape and sound....23
11. The picture path....24
11.1 Capture....24
11.2 Tear-free reads....24
11.3 To the browser....24
12. The host side: daemon, protocol and page....25
12.1 Register access: regs32.c....25
12.2 The daemon and its simulator....25
12.3 The pump....25
12.4 A bring-up probe built into the product....25
12.5 The web protocol....26
12.6 The page....26
13. Delivery: device tree, build, package, installer....26
13.1 The device-tree overlay....26
13.2 Building the gateware....27
13.3 The package and the installer....28
14. Performance and resources....28
15. Lessons learned....29
16. What is verified and what is not....31
17. A checklist for other projects....31
Appendix A. Register map....32
Appendix B. Gateware history....33
Appendix C. Sources and licences....33

1. The problem and the constraints

1.1 The board

The BeagleV-Fire carries a PolarFire SoC MPFS025T: a RISC-V multicore complex (one E51 monitor core and four U54 application cores running Linux) and an FPGA fabric of 22,956 four-input LUTs and 84 large block RAMs (LSRAM, 20 Kbit each). The only things the board gives the fabric are a clock, some DDR through the processor subsystem, and **the processor/fabric boundary itself**: memory-mapped windows in the RISC-V address map (the FIC) and interrupt lines back.

Table 1. What a retro computer needs and what the board offers.

Wanted by a retro computer	Provided by the BeagleV-Fire
A display	Nothing. No video connector is wired to the fabric.
A keyboard, joysticks	Nothing wired to the fabric. USB belongs to Linux.
A speaker	Nothing wired to the fabric.
Floppy drives, tape, hard disk, SD card	Nothing the fabric can reach. Storage belongs to Linux.
RAM and ROM	Block RAM on the chip (small) and DDR through an AXI port (large, variable latency).
A clock	50 MHz bus clock (PCLK), PLLs available; the machines run on a 42 MHz derived clock.
A way to talk to software	The fabric interconnect (FIC): a memory-mapped window and 24 interrupt lines.

So the only thing the board offers is the boundary itself, and everything else has to be built on top of it. That observation is the whole design: **if Linux can see the fabric as memory, Linux can be the monitor, the keyboard, the speaker and the disk drive**, and a web server is already running on it.

1.2 Requirements we set ourselves

1. A machine is a wrapper around an existing open core where one exists; we write a machine from scratch only when no usable core exists (the ZX Spectrum's ULA).
2. Zero special software on the user's computer. A browser is the only client.
3. A user installs it without a toolchain: a package with an installer.
4. It coexists with the board's stock software (its web server, Cockpit) and with other gateware designs.
5. Correctness before speed: 50 Hz video would be nice, a corrupted disk is not acceptable.
6. Switching machines is instant: no reflash, no reboot, no second bitstream.

1.3 Where this started

The project began as a port of one machine, the CP-300, whose FPGA core already existed for other boards (a DE0, the Analogue Pocket, MiSTer, Tang Nano). That port used about 78 % of the LUTs and 88 % of the block RAMs and ran the core unchanged, including the helper processor and firmware that serve disk images on the other boards. Everything in this paper that is not in that first port came from one ambition: use the same host interface for more machines. The ambition ran into the size of the chip quickly, and most of the interesting design work is the result of that collision (section 3).

2. Architecture overview

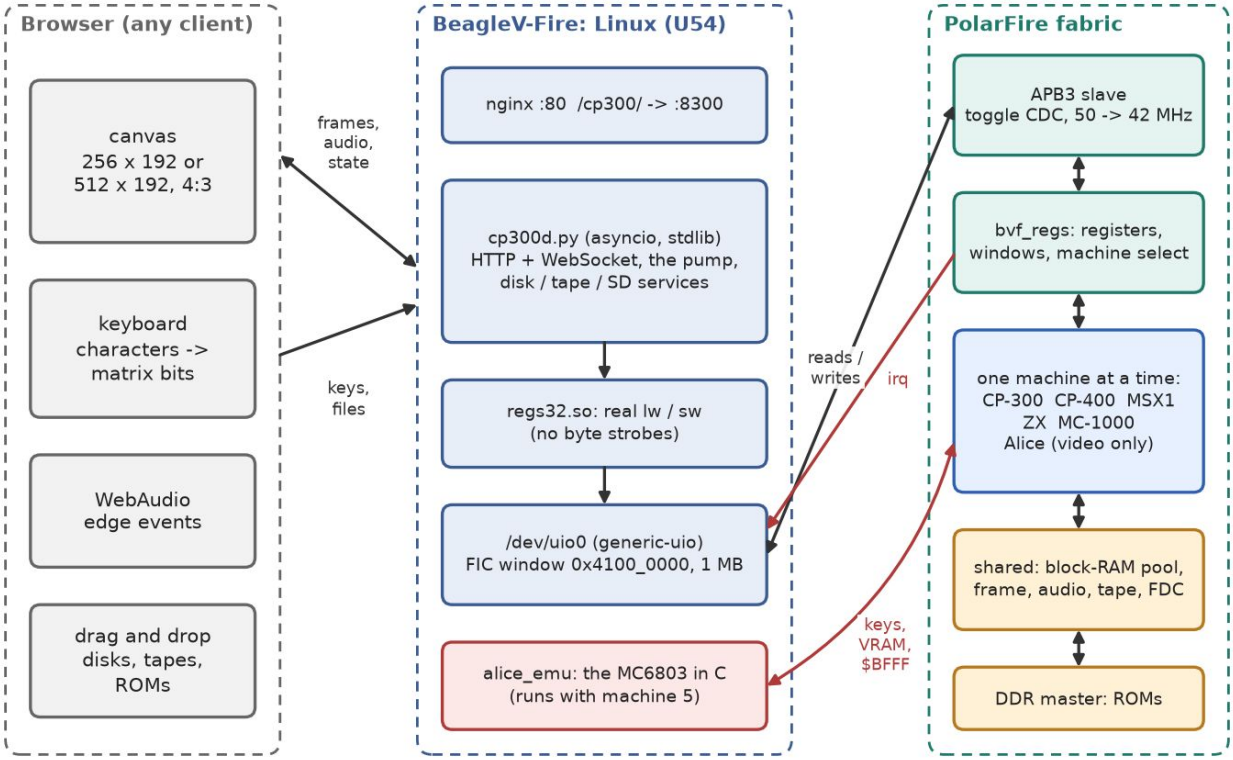


Figure 1. The whole system. The user-facing parts are on the left, the machines on the right; red arrows are the two paths that do not follow the usual request/response pattern (the interrupt, and the Alice's host-side CPU).

Six machines share one fabric design. Only one runs at a time; the register `CTRL` (bits 9:8 and 5:4) selects it, and the daemon changes it only while the core is held in reset. Four independent *flows* cross the boundary, and each got the mechanism that suits its nature:

Table 2. The flows across the processor/fabric boundary and the mechanism for each.

Flow	Direction	Nature	Mechanism
Control (power, reset, turbo, machine)	host → fabric	rare, state	registers
Keyboard	host → fabric	level state, 64 + 24 bits	registers, written whole
ROM load	host → fabric	one-shot bulk write	write-only window (word-addressed), or the DDR window for the MSX1
Screen	fabric → host	continuous stream, 12 or 24 KB per frame	frame RAM, read in bulk; consistency by counter (CP-300) or HOLD (others)
Audio	fabric → host	sparse, timing-critical events	timestamped edge FIFO, popped
Disk	fabric → host → fabric	request/response, *the fabric is the client*	REQ/ACK registers + 512-byte sector window + interrupt
Tape	both ways	pulse streams	play FIFO (host-fed) and record FIFO (timestamped edges)
Alice's CPU	host ↔ fabric	VRAM words and one latch out, key matrix in	window `0x60000`, register `0xA0`

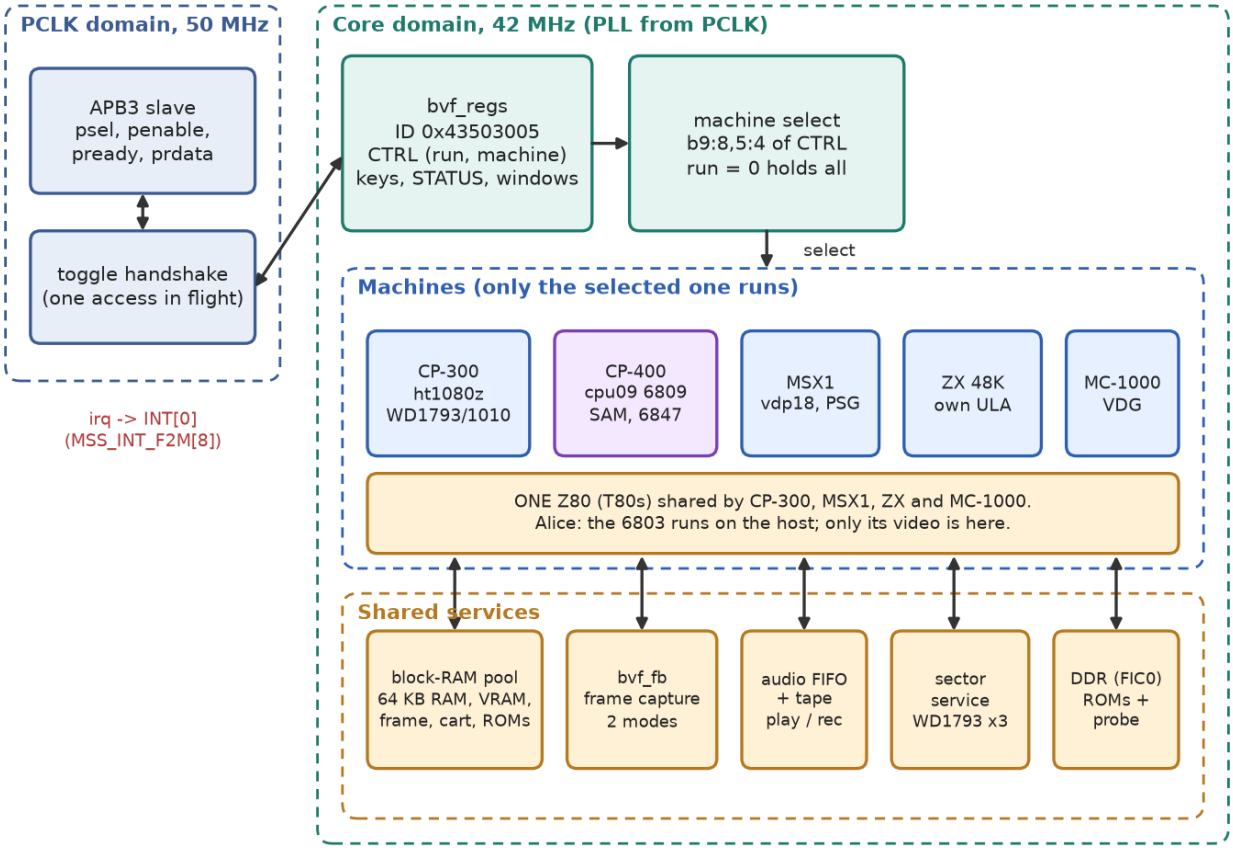


Figure 2. Inside the fabric. The bus runs at 50 MHz, everything behind the bridge at 42 MHz. One Z80 and one pool of block RAM serve several machines; the shared services (frame capture, audio, tape, sector service, DDR) are common to all.

The rest of the paper justifies this picture. Section 3 tells how it came about, section 4 lists the decisions, sections 5 to 7 describe the interface and the machines, sections 8 and 9 answer the two questions that shaped the design most (the Alice's CPU and the MSX video memory), and the later sections cover storage, picture, host software, delivery, measurements and lessons.

3. Architecture variants: what we built and what we threw away

The final design is not the first one, nor the best one in the abstract; it is the one that survived contact with a 23K-LUT chip. This section is the history of the variants, in the order we built them, with the reason each was replaced. The numbers are those recorded when each gateware was built (early rows are before place and route; they are indicative, not comparable to the last decimal).

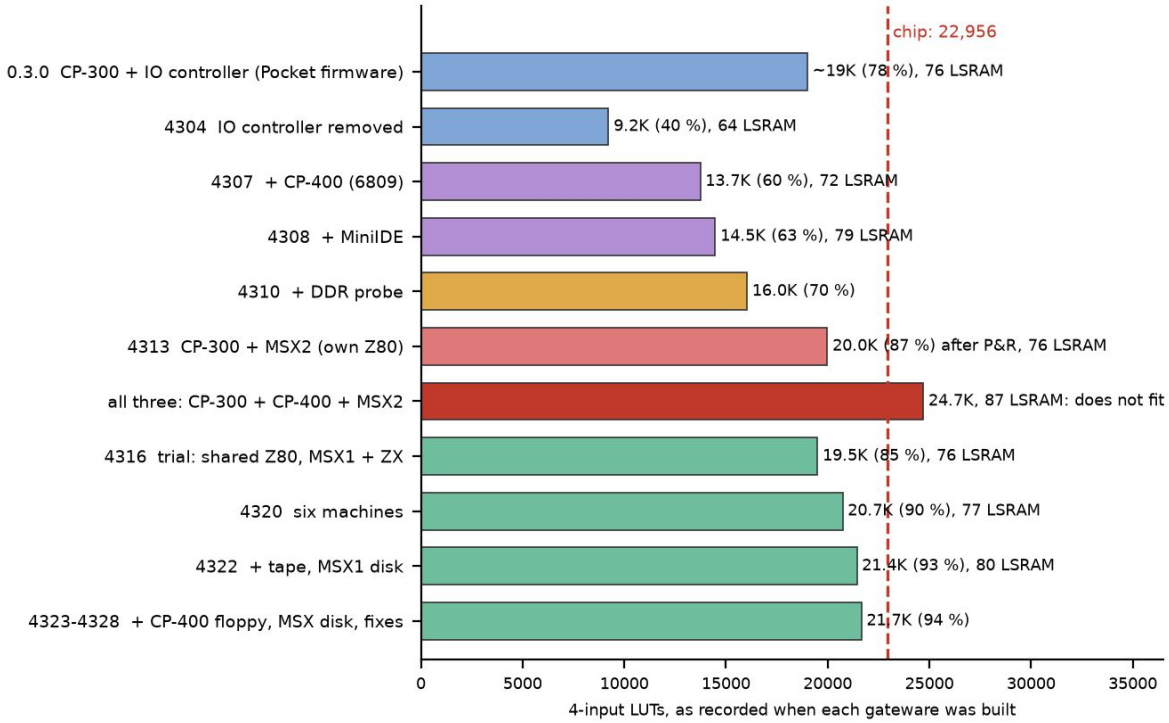


Figure 3. Logic use of the gateware generations. Green bars are the shared-Z80 line that became the final design; the red bars are the MSX2 branch that did not fit.

Table 3. The architecture variants in the order they were built.

Variant	What it was	Result and the reason it changed
V1 CP-300 + IO controller (0.2 - 0.3)	The Analogue Pocket core unchanged, with its helper Z80 ('IO controller') and 16 KB firmware that serves disk images. A new mailbox adapter put the firmware's file requests into the host register map.	Worked (TRSDOS boots, games play). The IO controller is **9.5K LUTs**, of which the firmware ROM is 7.2K synthesised as logic. It served only DMK/HDV images, which Linux can do itself. Replaced by V2.
V2 Host-served sectors (4304)	The IO controller and its mailbox removed. `bvf_fdc` hands the WD1793's and WD1010's sector requests to the daemon (`diskimg.py`, a port of the firmware's DMK code), one request per sector.	**9,226 LUTs (40 %)**: 9.5K LUTs and 12 block RAMs freed. Identical disk images verified against the firmware's C code. Cost: one Linux round trip per sector instead of per track (latency work, section 10).
V3 + CP-400 (4307, 4308)	The user's own CoCo 2 core (6809 `cpu09`, SAM, MC6847, two PIAs) as machine 1, with its own CPU; a MiniIDE (`bvf_ide`) for hard disks; the 64 KB RAM shared between the two.	13.7K, later 14.5K LUTs; 79 of 84 block RAMs. Works on the board. `cpu09` alone is 2.9K LUTs, the biggest block of the design. Not a Z80, so it cannot share the CPU of the other machines.
V4 DDR probe (4310, 4311)	An AXI master on FIC0 that reads and writes DDR and measures latency, to decide what could live in DDR.	Measured on the board: 256 ns typical read, 1.09 µs worst, 347 MB/s bursts (section 9). Defined the rule used ever after: *CPU-side memory may wait, video memory may not.*
V5 CP-300 + MSX2 (4313)	KdL's OneChip MSX (V9958 VDP, 128 KB VRAM) with its own Z80, VRAM packed into 56 block RAMs, main RAM and ROMs in DDR, a block-RAM pool shared with the CP-300.	Simulated and built (**20.0K LUTs**, 76 block RAMs) but not tried on a board. With the CP-400 added it needs **24.7K LUTs and 87 block RAMs**: it cannot fit (22,956 and 84). Replaced by V7.
V6 Cached DDR VRAM (abandoned)	A 128 KB VRAM in DDR behind a small cache, to free 56 block RAMs and about 3.4K LUTs.	Even with the cache the three-machine design stopped at 24.07K LUTs, and VDP blitter commands would run several times slower than on real hardware. Abandoned; branch kept.

Variant	What it was	Result and the reason it changed
V7 One shared Z80, MSX1 + ZX + MC-1000 (4316 - 4328)	The MSX2 dropped (2026-11-01). The CP-300, an MSX1 (TMS9918A, 16 KB VRAM), a ZX Spectrum 48K and an MC-1000 share **one T80s**; one block-RAM pool serves all machines; the MSX1's ROMs live in DDR behind a cache.	**21,659 LUTs (94.35 %)** in the final gateware, timing met. Six machines; the margin is 1.3K LUTs. Section 7.
V8 Alice with a CPU on the host (4320 onwards)	The Matra Alice's MC6803 is emulated in C on Linux; the fabric keeps the video (the MC-1000's VDG), the VRAM and the sound edge path.	About 60 LUTs and no block RAM for a seventh machine, tested in simulation; not yet run on the board. Sections 7.8 and 8.

Three patterns repeat in that table and they are the real design rules of the project:

- **Share what a machine does not use at the same time.** One Z80, one RAM, one set of block RAMs, one floppy controller, one frame path, one audio path, one tape path. Only one machine runs, so a block that is idle in one machine is free for another.
- **Move anything that is not real-time to the host.** The IO controller's firmware, the disk images, the file formats, the cassette tapes, the ROM files, and in the end the Alice's whole CPU. The fabric keeps what has hard timing: video generation, sound edges, bus cycles.
- **Measure before moving memory.** The DDR probe was built before any machine was put in DDR; its numbers (and not our intuition) decided that ROMs may be in DDR and VRAM may not.

4. Key decisions and the reasons

Each entry: what we chose, what we rejected, why. D1 to D10 are the decisions of the first port and still hold; D11 to D15 came with the other machines.

D1. Wrap the cores; keep the shared RTL almost untouched

The CP-300 core, the WD1793 and WD1010 controllers and the T80 are used as they are on the other ports. The BeagleV-Fire adds glue in `rtl/`. Three small, deliberate changes were needed in the shared core: the Z80 can be external (`EXT_CPU`, so that one T80 can serve several machines), the WD1793 has a second bus for the other machines' floppy cards, and a `sec512` input (512-byte sectors for the MSX). The CP-300 core files are vendored in `cp300core/` in this repository. **Cost:** no BeagleV-specific optimisation inside the core.

D2. The browser is the whole front end

Rejected: a bare-metal framebuffer driver (no connector to drive); a local Qt/SDL application (needs a display on the board); a VNC wrapper (a second protocol stack, poor audio). *Chosen:* a web page. The board already runs a web server and is reached over a network; browsers give a canvas, keyboard events, WebAudio, file drag and drop and a 4:3 layout engine for free, on any client OS, including a phone.

D3. One memory-mapped window, no kernel driver

Rejected: a custom kernel module (fragile against the board's exact kernel); a network path to the fabric; DMA descriptors. *Chosen:* a **generic UIO device**: the device-tree overlay declares a 1 MB region with `compatible = "generic-uio"`, the kernel exposes it as `/dev/uio0`, and an ordinary process maps it. UIO ships in the stock kernel. **Cost:** programmed I/O; a full frame is still read in about 1.5 ms.

D4. An interrupt wakes the host for disk requests; a timer drives the rest

The fabric raises a level interrupt for a pending sector (or FreHD, IDE) request. It is wired to bit 0 of the cape's `INT[23:0]` bus (`MSS_INT_F2M[8]`), which the overlay declares as PLIC source 126. The daemon unmasks it only

after answering, so with a level interrupt nothing is missed. A timed wake-up (0.5 ms after disk work, 4 ms idle) stays for picture and audio, and as the fallback on gateware without the interrupt.

D5. Frames are pulled and validated, not locked

The machine's timing must never depend on Linux. The CP-300's capture double-buffers and publishes a frame counter; the host brackets its bulk read with two counter reads and discards the copy if the writer lapped it. The other machines use a single buffer with a host-set HOLD bit (section 11).

D6. Audio is sent as edges with microsecond timestamps

Retro sound is one bit toggled by software. Sampling it quantises the timing and wastes bandwidth when silent. The fabric pushes `{level, time}` on every change; the browser rebuilds the waveform at its own sample rate. Silence costs nothing, timing is exact to 1 µs, and host latency only delays the sound without distorting it.

D7. The host is the disk; the fabric is the client

When the emulated CPU reads a sector, the floppy controller asks the host through REQ/ACK registers; Linux answers from an image file. All file handling (formats, persistence, download) is Python, testable without hardware. Since V2 this is one sector per request; since V3 three different controller front ends share it (section 10).

D8. The window is a 32-bit bus with no byte strobes

Discovered the hard way (L1). All access goes through a ten-line C helper that issues real `lw`/`sw`; no layer above it assumes a byte store works.

D9. One bridge from a 50 MHz bus to a 42 MHz core, with a toggle handshake

A four-state toggle handshake moves exactly one register access at a time between the domains; one mechanism covers every register and every RAM window.

D10. Ship a package with an installer, not a build recipe

The gateware needs Libero SoC 2026.1 (unofficial for this framework) and a pinned bootloader; no end user should repeat that. The package holds the bitstream, the overlay, the daemon, the page, the service and an installer that is idempotent and asks for explicit acceptance before it touches the flash (section 13).

D11. One Z80 for every Z80 machine

A T80s costs 2.3 to 2.6K LUTs. Four copies (CP-300, MSX1, ZX, MC-1000) would be about 10K LUTs for hardware of which only one is ever awake. The CPU is one block; its address, data and control buses and its clock enable are multiplexed to the selected machine (section 7.1). *Cost:* every machine must fit the T80's bus-cycle shape (writes complete when /WR ends; the interrupt-acknowledge cycle must read `$FF`); machine-specific timing is expressed through the clock enable.

D12. One block-RAM pool for every machine

The 64 KB RAM, the frame buffers, the CP-400's ROMs and the cartridge ROM are blocks of one array (`bvf_pool`), assigned per machine (section 7.2). The MSX1's VRAM is the CP-400's ROM blocks reused. *Cost:* the mux logic (a pool is not free: block-RAM interface logic is about 36 LUTs per block) and the rule that nothing may use a block while another machine runs.

D13. RAM and VRAM on the chip; ROMs in DDR behind a cache; CPU-side waits only

The DDR probe gave 256 ns typical and 1.09 µs worst-case reads. A Z80 memory cycle at 3.58 MHz is 840 ns, so a read fits in one cycle and the rare slow one is a WAIT. A video chip fetching every few dot clocks (93 ns)

cannot wait. So: RAM 64 KB (no wait) and VRAM on the chip; the MSX1's ROMs (up to 96 KB) in DDR with an 8 KB read cache. Details in section 9.

D14. A CPU that is slow and simple belongs on the host

The Alice's MC6803 needs 0.2 to 0.3 million instructions per second. A fabric core would need logic the chip does not have; the host has cycles to spare. Section 8.

D15. Reset and power are different things

A real RESET keeps the RAM (the CoCo's ROM even looks for a warm-start flag; a game survives it). Only switching off clears it. The register `CTRL` has a RESET pulse and a COLD pulse; COLD starts `bvf_ramclr`, which writes zero to all 65,536 bytes (1.6 ms at 42 MHz) with every machine held, and then the machine starts from nothing. The page has Reset and Power buttons accordingly.

5. The platform: what the BeagleV-Fire gives you

The integration points transfer to other SoC-FPGA boards (Zynq, Cyclone V SoC, Agilex, other PolarFire SoC boards); the device details do not.

- **The MSS** (microprocessor subsystem) holds the RISC-V cores, the memory controller and the buses; its configuration is *generated* (`pfsoc_mss`) and is not part of the fabric design.
- **The FIC** exposes the fabric as address ranges in the processor's physical map and carries fabric interrupts back. In the board's framework the fabric bus is `fabric-bus@40000000` in the device tree, and a "cape" receives an **APB3 slave** at `0x41000000` (16 MB of window; this project uses the first 1 MB) and 24 interrupt lines. A second port, **FIC0's AXI4 initiator**, reaches DDR: 64-bit data, 125 MHz.
- **The gateware framework** (BeagleBoard.org) is a Libero Tcl build system that assembles the MSS, the stock infrastructure and a selectable cape option. Adding a design means adding a cape directory; `gateware/make_cape.py` does this.
- **Boot and flashing.** The bootloader (HSS) and the bitstream live in SPI flash. Reprogramming the fabric rewrites that flash through `change-gateware.sh`, **with no fallback copy**. This single fact shaped the installer and the disclaimer more than anything else.

> **Transferable rule.** On any SoC-FPGA find out first (1) how the CPU addresses fabric registers and how wide and strobe-capable that path is, (2) how the fabric's identity reaches the OS, (3) what happens when the fabric is reprogrammed under a running Linux, and (4) how you get back if you break the boot path.

6. The FPGA/CPU interface

6.1 The address map

Table 4. The register window. Unmapped windows read as zero.

Offset	Window	Use
`0x00000`	registers	ID `0x43503005`, CTRL, STATUS, keys, FRAME, AUDIO, REQ/ACK/DRIVES, IDE, DDR probe, FreHD, ALICE, tape, DEBUG (appendix A)
`0x10000`	ROM load (write only)	word n of a ROM image at `0x10000 + 4n`, first byte in bits 31:24 (the CP-300's system ROM and character generator, the ZX ROM, the CP-400's ROMs, the MC-1000 ROM, the Alice ROM goes to the emulator)
`0x20000`	sector buffer (512 B)	the floppy / hard-disk controllers' sector in transit, little-endian words

Offset	Window	Use
`0x30000`	frame RAM (read only)	buffer 0 at `0x30000`, buffer 1 at `0x34000` (CP-300); one 24 KB buffer for the colour machines
`0x40000`	IDE sector buffer	the CP-400's MiniIDE sector, 512 bytes
`0x50000`	FreHD data buffer	word n = byte n, 256 words
`0x60000`	Alice VRAM (write only)	word n = VRAM bytes 4n..4n+3, n = 0..1535

Design rule used throughout: *one register file is the only interface; the other windows are bulk memories hanging off the same address decoder.* A new peripheral costs one more `case` arm and one more window, never a new bus. The `ID` register is the contract's version: it has had five values (`...01` IO controller mailbox, `...02` one machine and no IO controller, `...03` machine select and frame hold, `...04` the Alice, `...05` tape), and the daemon refuses to run against a gateware with a different one.

6.2 Crossing from the 50 MHz bus to the 42 MHz core

The APB3 slave runs on PCLK; the machines run on a PLL-derived 42 MHz clock. Rather than synchronise every signal the bridge synchronises a single bit:

```
PCLK domain                                  core clock domain
-----------                                  -----------------
psel & penable & !busy -> latch addr/wdata/wr
                       -> req_tog flips   --2FF-->  rq2 != rq3 ?  -> drive the register bus for 1 clock
busy = 1                                            read: wait 2 clocks for data (registered RAM outputs)
ack2 == req_tog  <--2FF--  ack_tog flips  <-------- done
-> prdata = r_rdata, pready = 1 (1 clock), busy = 0
```

- **One flag in flight per direction**, so single-bit toggles through a two-flop synchroniser are safe. The *data* is not synchronised: it is stable for the whole handshake by construction.
- **PREADY is used.** APB3 permits wait states, so a 12 KB frame is read with ordinary loads and the CPU simply waits. The framework's template cape exposes only `PRDATA`; `make_cape.py` adds `PREADY` and `PSLVERR` to the cape's ports.
- **The cost** is several clocks of each domain per access: irrelevant for control and mailbox traffic, and for the frame it defines the 1.5 ms bulk read.
- **A known gap:** there is no bus timeout. If the 42 MHz clock were absent the CPU would hang on the access.

6.3 Control, the start sequence and the three hold states

`CTRL.run = 0` holds every machine in reset. The daemon's power-on sequence is the contract the registers were designed around:

```
1. CTRL.run = 0                          # hold everything (machine select changes only here)
2. load the machine's ROMs               # ROM window (word packing) or, for the MSX1, the DDR window
3. publish DRIVES, IDE_CFG               # drive ready / write-protect flags; kept while run = 0
4. CTRL.run = 1 with COLD                # RAM cleared (1.6 ms), then the machine starts from nothing
```

Step 3 is before step 4 because the drive flags are read by the machine at its first access; the first port got this order wrong (L2). The hold conditions are nested and keeping them straight is most of the debugging:

```
core_reset (CP-300) = ~run | ~pll_locked | rst_pulse | clr_active | ~m300     # one such term per machine
regs reset          = ~pll_locked                                            # so that run can be written
audio / tape reset  = ~run
frame capture       = no reset: it captures whatever the machine's video produces
```

6.4 The read-latency convention

`bvf_regs` documents one convention that every block follows: **read data is valid two clocks after the read strobe**. RAM outputs are registered (one clock) and the window mux is selected by a delayed copy of the address,

so the mux picks the window of the previous request. Fixing this once, in a comment at the top of the register file, saved reasoning per block about when data appears.

7. The machines

Each machine is a small design around a core that already existed; the sections below describe how each one is attached to the shared infrastructure and what had to change. Sources and licences are listed in appendix C.

7.1 One Z80 for four machines

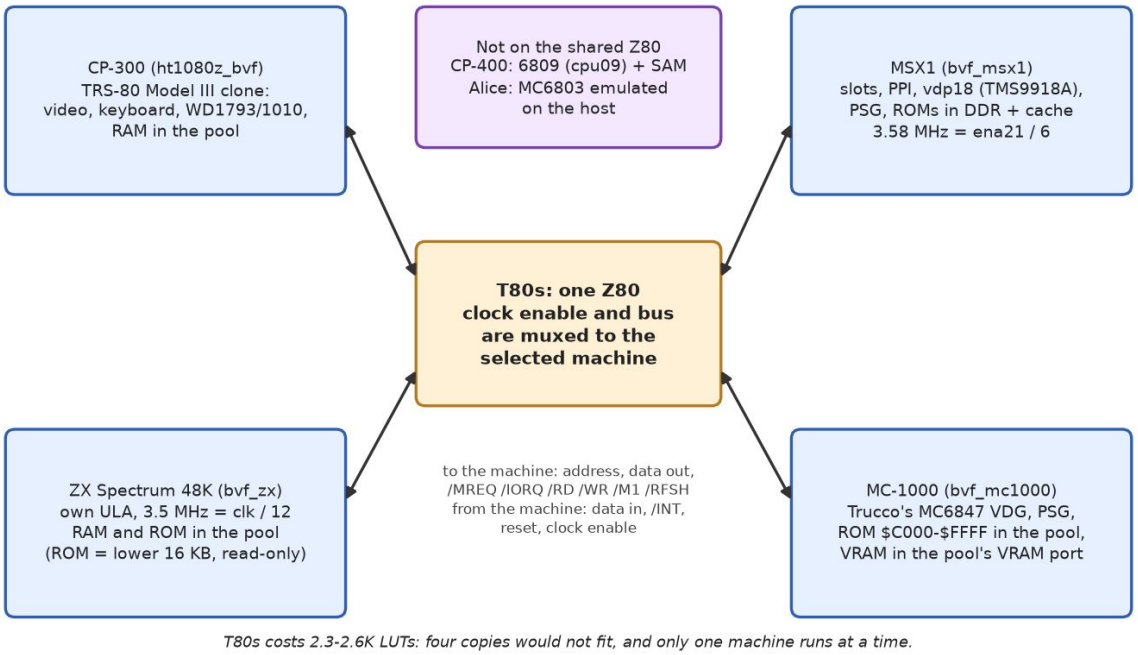


Figure 4. The shared Z80. The T80s is a single instance; its buses, interrupt, reset and clock enable are multiplexed to the selected machine. The CP-400 has its own 6809 and the Alice has no CPU in the fabric.

The CP-300's core (derived from the HT1080Z) was written with its Z80 inside; a generic (`EXT_CPU`) moves the CPU outside as `ht1080z_bvf.vhd`. Each machine then supplies a clock enable and receives the CPU's bus:

Table 5. Clocking of the machines on the shared Z80.

Machine	Effective Z80 clock	What the machine adds
CP-300	its own enable (the HT1080Z's)	video, keyboard, WD1793/WD1010, FreHD interface, speaker
MSX1	3.58 MHz = `ena21` / 6, with `ena21` = 45/88 of 42 MHz (21.477 MHz)	slot logic and PPI, TMS9918A on `ena21` / 2, PSG, WAIT when a ROM byte is not in the cache
ZX Spectrum	3.5 MHz = 42 MHz / 12	own ULA; no contention, no floating bus
MC-1000	3.58 MHz	MC6847 VDG, PSG, keyboard through the PSG's ports

Two bus-cycle rules of the T80s caused real bugs and are worth stating. **A write is acted on when /WR ends**, with the address and data captured while it is low (the T80 puts them out after /WR falls). **During the interrupt-acknowledge cycle (M1 with IORQ) the bus must read $FF and memory must not answer**, or the vector the CPU takes is wrong.

7.2 One block-RAM pool

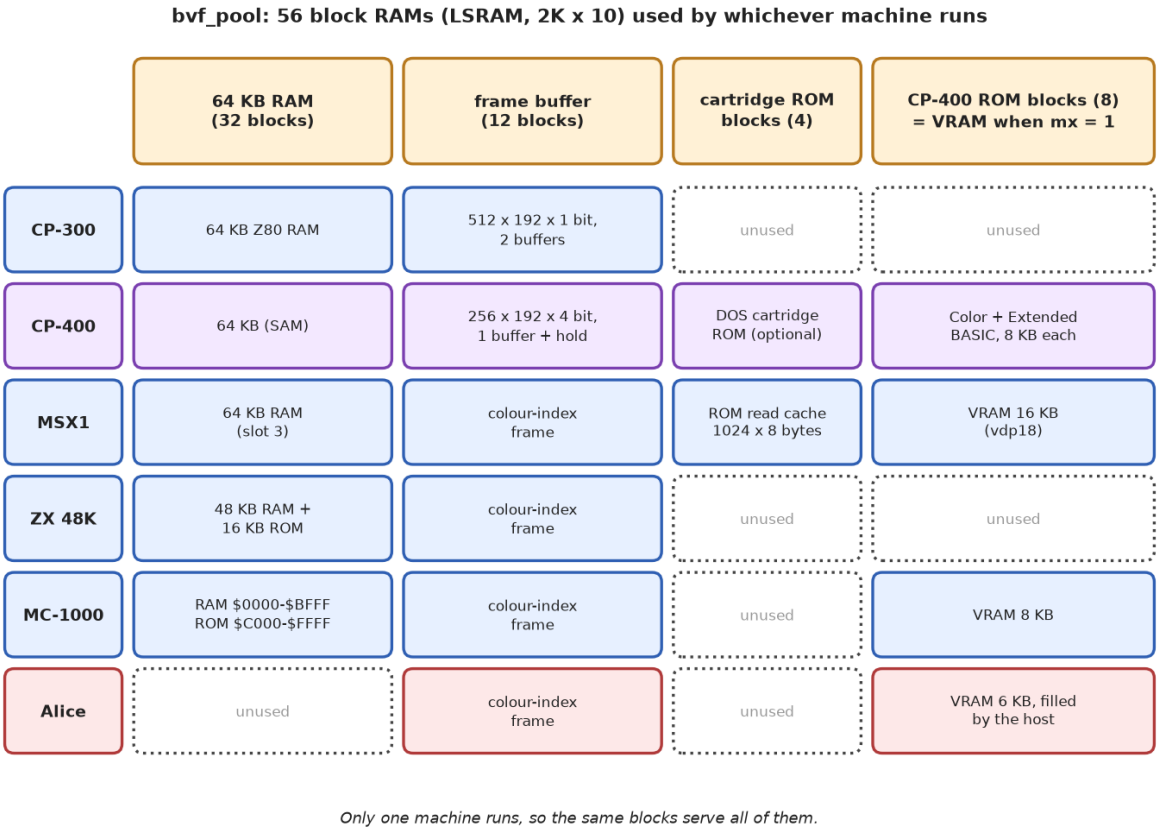


Figure 5. `bvf_pool`: 56 of the chip's 84 block RAMs, assigned differently by each machine.

The 64 KB RAM has one read address and one write address (separate ports), with one clock of read latency; whoever owns the address mux owns the memory. The frame buffer is three sets of four blocks. The CP-400's two 8 KB ROMs are eight blocks that every other machine reuses: as the MSX1's 16 KB VRAM, the MC-1000's 8 KB VRAM and the Alice's 6 KB VRAM. The cartridge blocks are the CP-400's DOS cartridge and the MSX1's ROM read cache. A power-on clear (`bvf_ramclr`) writes zero to the whole 64 KB while the machines are held; it skips the ZX Spectrum's ROM (the lower 16 KB) and the MC-1000's ROM (the upper 16 KB), which live in the same array.

7.3 CP-300 (machine 0)

A Z80 machine with a 512 × 192 monochrome display, a 64-position keyboard matrix, a one-bit (three-level) speaker, a WD1793 floppy controller and a WD1010 hard-disk controller. The system ROM (16 KB) and the 2 KB character generator are loaded by the daemon (they are copyrighted: never part of a bitstream or a package). The FreHD hard-disk interface (`bvf_frehd`, ports C2 to C5) sits on the core's expansion bus and presents the board's PIC firmware's command set over a folder on the SD card (`frehd.py`): open, read, seek, directory, mount an `.hdv`. The core's own cassette port is bridged to the shared tape unit.

7.4 CP-400 / CoCo 2 (machine 1): a 6809 machine

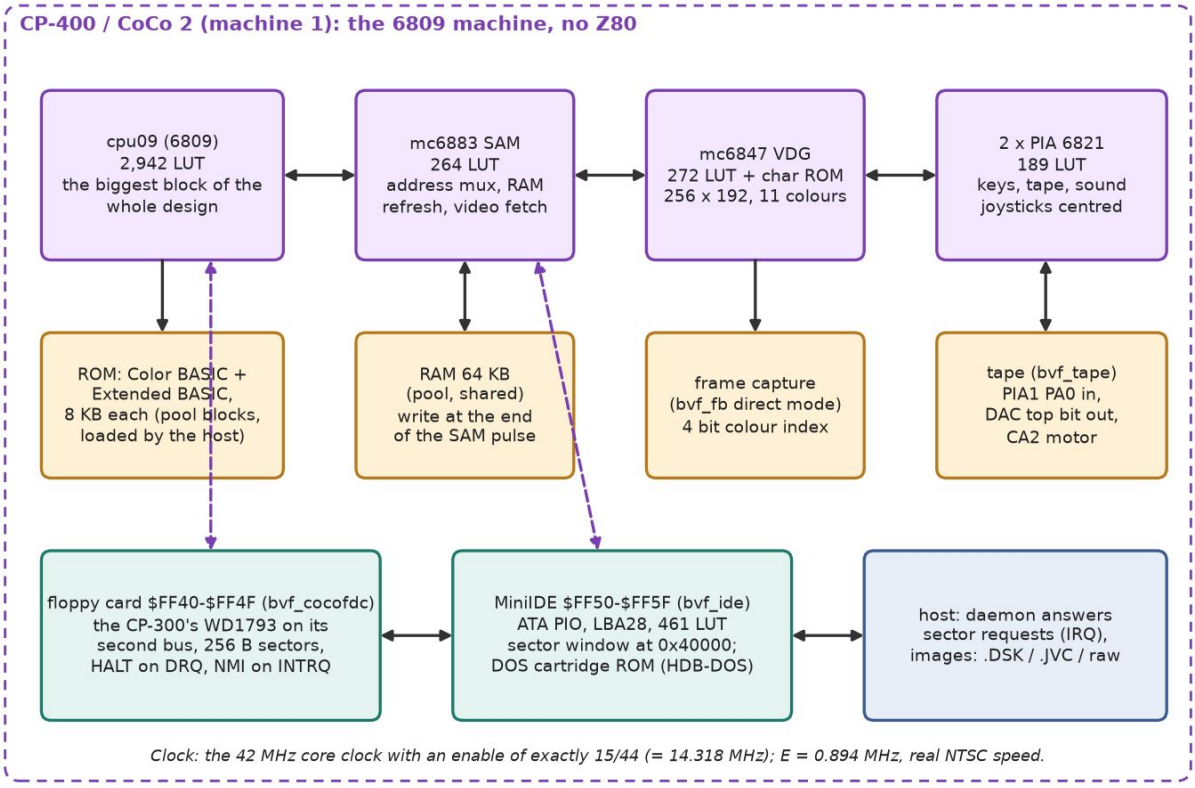


Figure 6. The CP-400: its own CPU (`cpu09`, a John Kent 6809), the SAM, the MC6847 and two PIAs, on the shared RAM, frame capture, tape, floppy controller and IDE.

The CP-400 is the user's own CoCo 2 core (Victor Trucco's Multicore core, based on Mark McDougall's PACE), reduced to its board-independent part (`cp400_core.vhd`) and re-clocked. The original runs at 57.27 MHz with a 14.318 MHz enable; here the 42 MHz core clock with an enable of **exactly 15/44** of it, so the machine has real NTSC speed. Two details cost a debugging day each: the CPU enable must be combinational at the first clock of E (a registered enable was two clocks late and corrupted every 15th RAM write), and the cartridge line must be held high when no cartridge is present (with the original's `CART = Q` the ROM took FIRQs and restarted forever).

Storage and the rest: a **MiniIDE** (`bvf_ide`, ATA PIO, LBA28, 461 LUTs) replaces the original's OCIDE and SD card at `$FF50-$FF5F`; the daemon serves its sectors; an HDB-DOS cartridge ROM is optional. A **floppy controller card** (`bvf_cocofdc`, `$FF40-$FF4F`) puts the CP-300's WD1793 on the CP-400's bus with the CoCo's control register, HALT on DRQ and NMI on INTRQ (section 10). Tape goes through PIA1 PA0 / the DAC / CA2. Joysticks are centred; serial is not implemented.

7.5 MSX1 (machine 2)

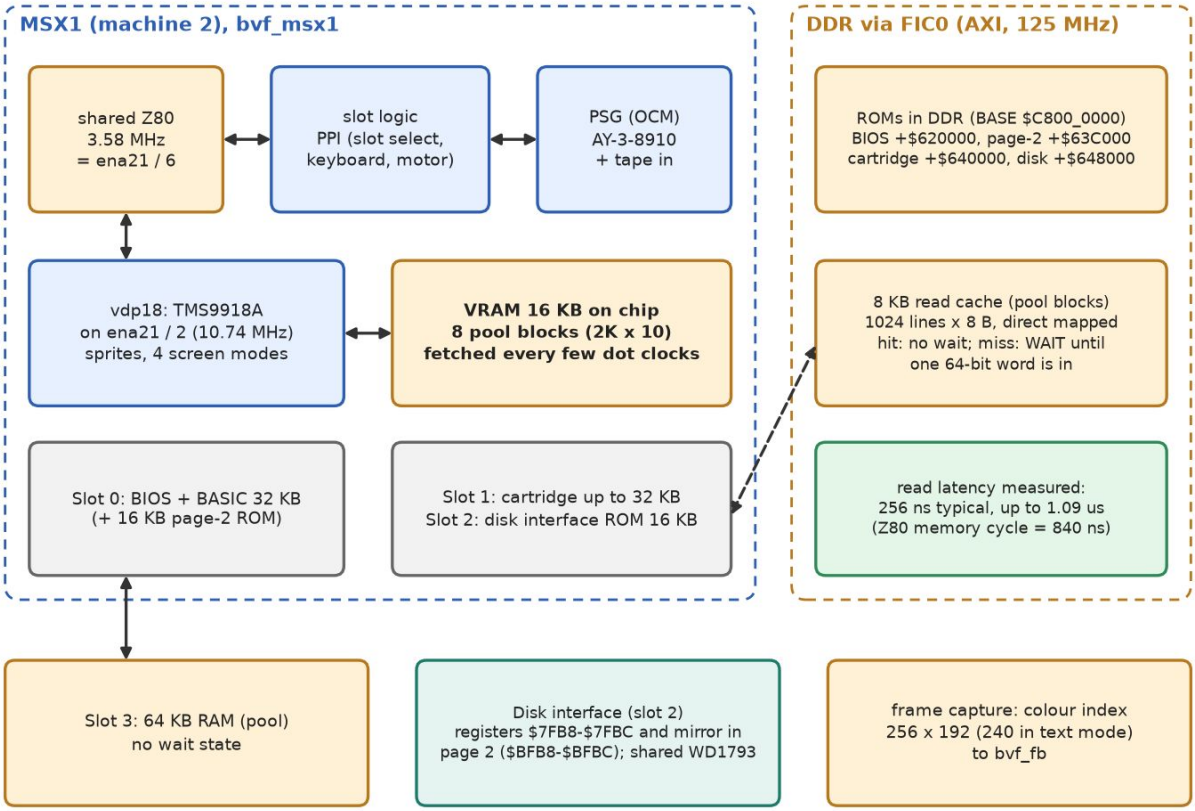


Figure 7. The MSX1. RAM and VRAM are on the chip; the ROMs (and the disk interface ROM) are in DDR behind an 8 KB read cache.

The MSX1 is built from the TMS9918A (vdp18 by Arnim Laeuger, from the MSX1fpga core) and the PSG of KdL's OCM-PLD, with the slot logic written here. The slot map: slot 0 the 32 KB BIOS + BASIC (and a 16 KB ROM in page 2 for C-BIOS's logo); slot 1 a cartridge of up to 32 KB at `$4000-$BFFF`; slot 2 the disk interface (below); slot 3 the 64 KB RAM, which is the pool's (no wait state). Keyboard: the page sends an 88-bit matrix (`KEYS_LO/HI` and `KEYS2`). The tape input is PSG register 14 bit 7; the motor and output are on PPI port C.

ROMs in DDR. The BIOS, cartridge and disk ROMs sit in the reserved 32 MB region at `$C800_0000`, loaded by the host while the machine is held (through the same AXI master the probe uses; while `CTRL.run = 0` the probe owns the bus). In front of them an 8 KB read cache (1024 lines of 8 bytes, direct-mapped, in the pool's cartridge blocks): a hit answers without a wait state; a miss asserts WAIT on the Z80 until one 64-bit DDR word is in. The tags are cleared after every reset. This is decision D13 at work: the Z80 can wait, so ROMs can be far away.

The video memory is not in DDR. The VDP fetches VRAM every few dot clocks (a dot is 93 ns at 10.74 MHz) and cannot wait; its 16 KB are eight pool blocks (section 9).

The disk interface. The Gradiente Expert DD Plus's layout ("National" style) was decoded from the disk ROM's own accesses: WD1793 registers at `$7FB8-$7FBB` and a control/status register at `$7FBC`. The first implementation worked in simulation and then needed three corrections on real software (section 10 and L9).

7.6 ZX Spectrum 48K (machine 3)

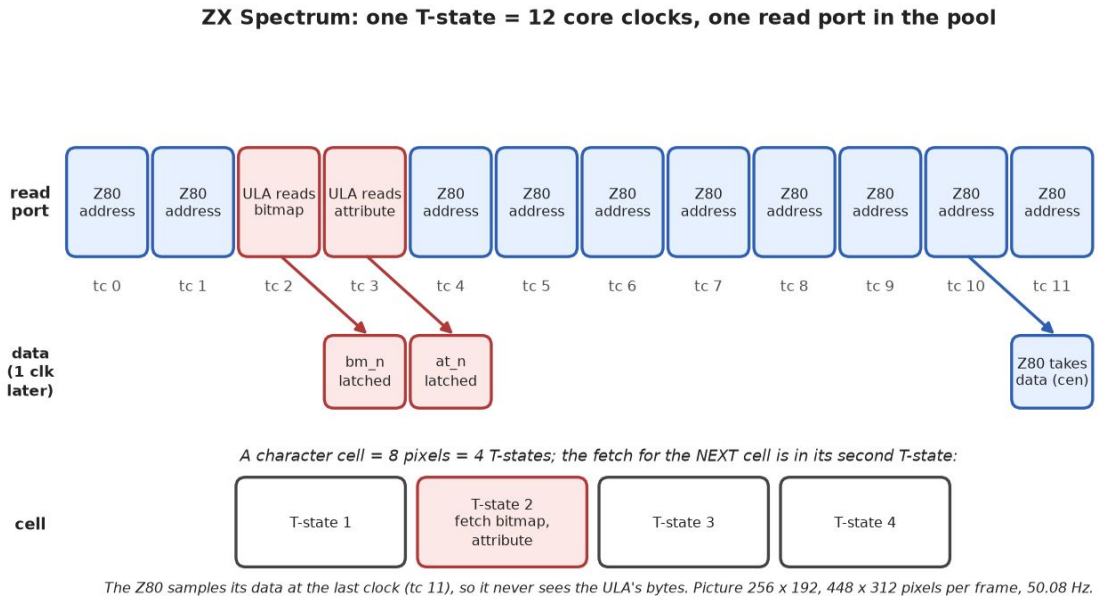


Figure 8. The Spectrum's ULA reads the screen through the same single read port as the CPU. Twelve core clocks make one T-state; the ULA owns the port for two of them.

There was no ZX core we could use on the shared Z80, so the ULA is written here (`bvf_zx`). The Z80 runs at 3.5 MHz (one T-state is 12 clocks of the 42 MHz clock, a pixel is six). Memory is the pool's 64 KB with the ROM in its lower 16 KB, written by the host while the machine is held and write-protected afterwards. The pool has **one read port**: in every T-state the CPU's address is on it, except at clocks 2 and 3 of the second T-state of a character cell, where the ULA reads the bitmap byte and the attribute byte of the *next* cell. The Z80 takes its data at clock 11, so it never sees the ULA's bytes. The picture is 448 × 312 pixels at 50.08 Hz; only the 256 × 192 window is captured, the border colour is an output of its own that is not shown. The interrupt is raised for the first 32 T-states of line 0. There is no memory contention and no floating bus: software that depends on either will differ from a real Spectrum. Keyboard half-rows, EAR (port `$FE` bit 6) and MIC (bit 3) are connected.

> **Open issue.** On two of the four most recent gateware builds (4326 and 4327) the Spectrum boots and runs its ROM but the picture is all black, while on 4325 and 4328 it is correct. The ZX logic is identical in all four, and the simulation of the same RTL shows the picture. With the debug word of 4327 the ULA's last attribute and bitmap fetches read `00` in all 32 samples while the Z80 sat in the ROM's keyboard loop. The cause is not found; it behaves like a placement-dependent hardware difference on the one-read-port arrangement. See L10.

7.7 MC-1000 (machine 4)

The CCE MC-1000 (Brazil, 1985) follows Victor Trucco's MC1000 core without the 80-column card: the shared Z80 at 3.58 MHz, the MC6847 of his core (renamed and given capture ports), the OCM's PSG, the 64 KB RAM at `$0000-$BFFF` and the 16 KB ROM at `$C000-$FFFF` (loaded by the host; after a reset it is read at every address until the first access to `$C000`). The 8 KB VRAM is the pool's VRAM port, at `$8000-$9FFF` while port `$80` bit 0 is 0; an access to it waits while the VDG shows the picture. The keyboard is read through the PSG's ports A and B, and the interrupt is an 8.8 µs pulse every 3.45 ms.

7.8 Matra Alice / Tandy MC-10 (machine 5): the CPU on the host

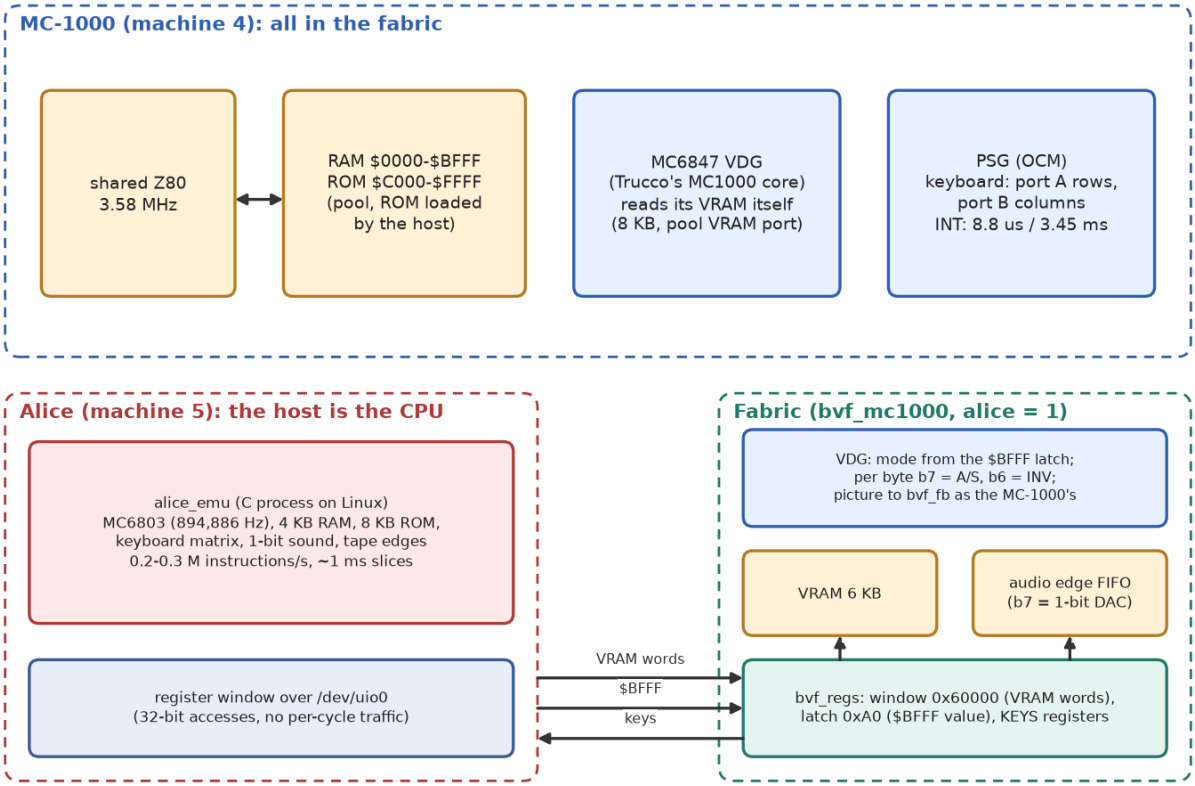


Figure 9. The MC-1000 and the Alice share the MC6847 VDG. For the Alice the CPU, RAM, ROM and keyboard are a C process on Linux; the fabric keeps the video, the VRAM and the sound edge path.

The Alice is a French clone of the MC-10 (1983) with a Motorola MC6803 and an MC6847. Its CPU is emulated in C (`alice/emu`, 93 hand-checked tests, runs the MC-10 ROM to the BASIC prompt); the fabric is the MC-1000's VDG with the Alice's per-byte attributes (`bvf_mc1000` with `alice = 1`), the pool's VRAM, the register `0xA0` (the `$BFFF` latch: mode, colour set, DAC bit) and the `0x60000` window. About 60 LUTs and no block RAM. Why this and not a CPU core in the fabric is the subject of the next section.

8. Why the Alice uses a soft CPU instead of an FPGA core

Five of the six machines have a CPU inside the fabric: the shared T80s (four machines) and `cpu09` (the CP-400). The sixth, the Matra Alice, has its Motorola MC6803 **emulated in C on the board's Linux**. This is the decision most likely to look odd in an FPGA project, so this section gives the whole argument: what the machine needs, what the alternatives would have cost, what crosses the boundary, and what remains unproven.

8.1 What the machine needs

Table 6. What the Alice needs.

Item	Value	Source
CPU	MC6803, 3.579545 MHz / 4 = 894,886 Hz, no on-chip ROM	MAME's `mc10.cpp` / `m6801.cpp`, checked by running the MC-10 ROM
Instruction rate	0.2 to 0.3 million instructions per second	arithmetic from the clock and the 6801's cycle counts
Memory	4 KB RAM at `$4000`, 8 KB ROM at `$E000`, on-chip RAM `$80-$FF`	MC-10 map
Video	MC6847 reading the RAM from `$4000`; per byte b7 = A/S, b6 = INV	same chip as the MC-1000's and the CP-400's

Item	Value	Source
Sound	one bit, `$BFFF` bit 7	software toggles it: timing matters
Keyboard	8 columns × 7 rows, scanned through port 1 and read from `$BFFF`	matrix in `alice/emu/alice.c`

8.2 The options

Table 7. The ways to give the Alice a CPU.

Option	Logic	Block RAM	What it takes	Verdict
A. An MC6803 in HDL, in the fabric	a 6800-family core with timer, ports and serial: of the order of 1.5 - 3K LUTs (estimate: the 6809 `cpu09` is 2.9K; no 6803 core was built)	RAM (4 KB) and ROM (8 KB) would fit in the shared pool	a new core to write or port, and to verify against a machine whose ROM behaviour we only know from MAME	**No room in the logic.** Only 1.3K LUTs are free at the end.
B. The host as bus master of the CP-400's SAM (the first draft)	small	none	the 6809 side is replaced by writes through the register window at the SAM's timing; changes to `cp400_core`; the SAM fetches video every 14 MHz tick	Rejected: needs the exact SAM timing and touches a core that works.
C. CPU on the host, **video in the fabric** (chosen)	**about 60 LUTs**	none (the VRAM is the pool's)	a C emulator, a VRAM window, one register, the MC-1000's VDG with two attribute changes	**Chosen.**
D. As C, but bare metal on one U54 hart (AMP)	same as C	none	a boot-chain change (HSS payload or remoteproc); jitter drops further	Later, if C's sound jitter is not good enough.

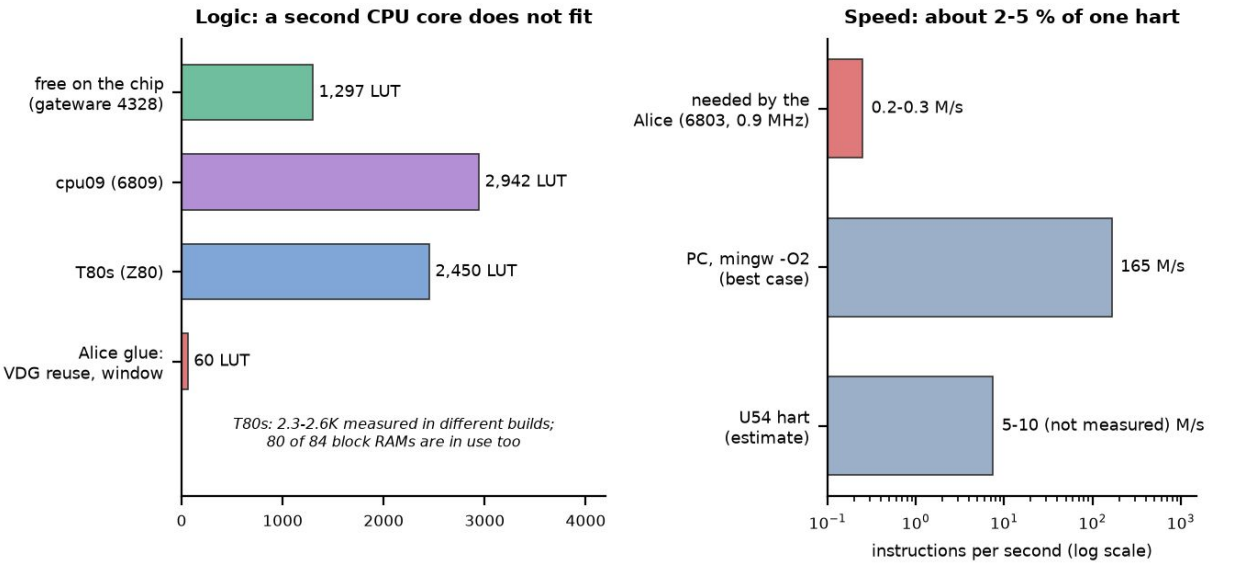


Figure 10. Left: the logic budget. After the six-machine gateware 1,297 LUTs are free; the smallest CPU already in the design costs 2.3 to 2.6K. Right: the host's load. The PC figure is measured (best case, a spinning ROM); the U54 figure is an estimate and has not been measured.

8.3 What crosses the boundary, and what does not

The whole argument for C rests on one number: **nothing crosses the FIC per 6803 cycle.** The emulator keeps the CPU, the RAM, the ROM and a shadow of the video RAM in its own memory. After each slice of about 1 ms of

machine time it writes the VRAM words that changed (a full 6 KB redraw is 1,536 posted writes of 32 bits) and, right after the instruction that stored it, the `$BFFF` latch. The key matrix it reads from the existing `KEYS` registers once per slice.

Table 8. The Alice's interface to the host.

Direction	What	How
host → fabric	the video RAM, `$4000-$57FF`, 6 KB	window `0x60000`: word n = bytes 4n..4n+3; the gateware writes the four bytes into the pool's VRAM port one per clock (`ALICE` b8 = busy)
host → fabric	the `$BFFF` latch: DAC bit, CSS, A/G, GM0-2 / INT-EXT	register `0xA0`, written right after the instruction that stored it
fabric → host	the key matrix	`KEYS_LO/HI`, bit = 8 × column + row
fabric internal	sound	`{al_mode[7], 8'h00}` into the shared audio FIFO: the gateware timestamps the edge at the register write

The fabric half is the MC-1000's video: its MC6847 generates its own video addresses, so no part of the CP-400 core changes and no SAM timing is involved. Two things differ and are done by parameter: the Alice keeps its attributes in the video data (b7 = A/S, b6 = INV; the MC-1000 keeps INV in b7), and the VDG's mode comes from the latch instead of from an I/O port. The character ROM, semigraphics-4 colours and the frame path are shared.

8.4 Why this is better than it looks

- **It is testable before any gateware exists.** The emulator was developed and tested on a PC: 93 hand-computed checks (flags of every ALU group, 16-bit operations, MUL, DAA, stack and interrupt frames, timer flags, ports) and a sweep of all 256 opcodes against an independently written table. The MC-10 ROM reaches `MICROCOLOR BASIC 1.0 / OK` in 3 s of machine time and runs `PRINT 1+1`. A ROM that exercises most addressing modes (the 16 KB Alice 32 ROM) runs 5 s with 0 illegal opcodes.
- **The fabric half is small enough to simulate completely.** ModelSim runs the real `bvf_core_bus`, the pool, the VDG and the frame capture from a VRAM + mode image produced by the emulator: the MC-10 prompt, semigraphics-4 and RG6 graphics appear (`sim/alice_*.png`). The real-time runner against a register array on the PC produces a VRAM image byte-identical to the harness's after 3 s.
- **The speed is not a question.** 165 million instructions per second on a PC in the best case (500 × real time); about 2 to 5 % of one U54 hart by estimate.
- **The code is structured for the other option.** The CPU file has no dependency (no malloc, no libc) and talks to its machine through bus callbacks; the platform layer is meant to shrink to four calls (VRAM flush, mode, DAC, keys; not yet written), so the same code could later run bare metal.

8.5 What is not known

- **Not run on the board.** The hardware mapping of the emulator (`mmap` of the UIO window) was not compiled on the PC (Windows has no `sys/mman.h`); the installer compiles it on the board.
- **Sound jitter.** The sound edge is timestamped when the register write arrives, so the jitter is that of one FIC write (a few microseconds) plus Linux scheduling of the emulator process. Pinning to a core with `SCHED_FIFO` is not done and the effect on a 1-bit sound has not been measured.
- **Field sync.** The MC-10 ROM ran to its prompt without the 6847's field sync on the CPU's interrupt; whether the Alice's ROM needs it is open.
- **The Alice 32 / 90** (EF9345 video) is a different machine and out of scope. The original Alice ROM has not been run; only the MC-10's (the same hardware).

When is a soft CPU the right answer? When (1) the CPU needs a few percent of one core, (2) its traffic to the rest of the machine can be batched (here: changed VRAM words and one latch, not a bus cycle), (3) the hard-real-time parts (video generation, sound edges) are separable and stay in the fabric, and (4) area is the scarce resource. All four held for the Alice. None would hold for the Spectrum, whose ULA steals memory cycles from the Z80 with clock-level timing.

9. Why the video memory must be on the chip, and what the MSX2 taught us

9.1 The rule: the CPU may wait, the video may not

Before any machine memory was moved off the chip we built a probe (`bvf_ddr`, V4): an AXI master on FIC0 that reads and writes DDR and measures its own latency. The board's numbers, with Linux running:

Table 9. The DDR probe on the board (gateware 4311).

Measurement	Result
ACLK (the AXI side)	125 MHz
One 64-bit read	32 ACLK = **256 ns** typical; minimum 26 (208 ns), **maximum 136 (1,088 ns)**
One 64-bit write	8 ACLK = 64 ns; average 186 ns over 1,000, maximum 520 ns
Bursts of 16 words	2.88 ACLK per word = 347 MB/s
Data integrity	1,000 words written, read back and compared: 0 errors

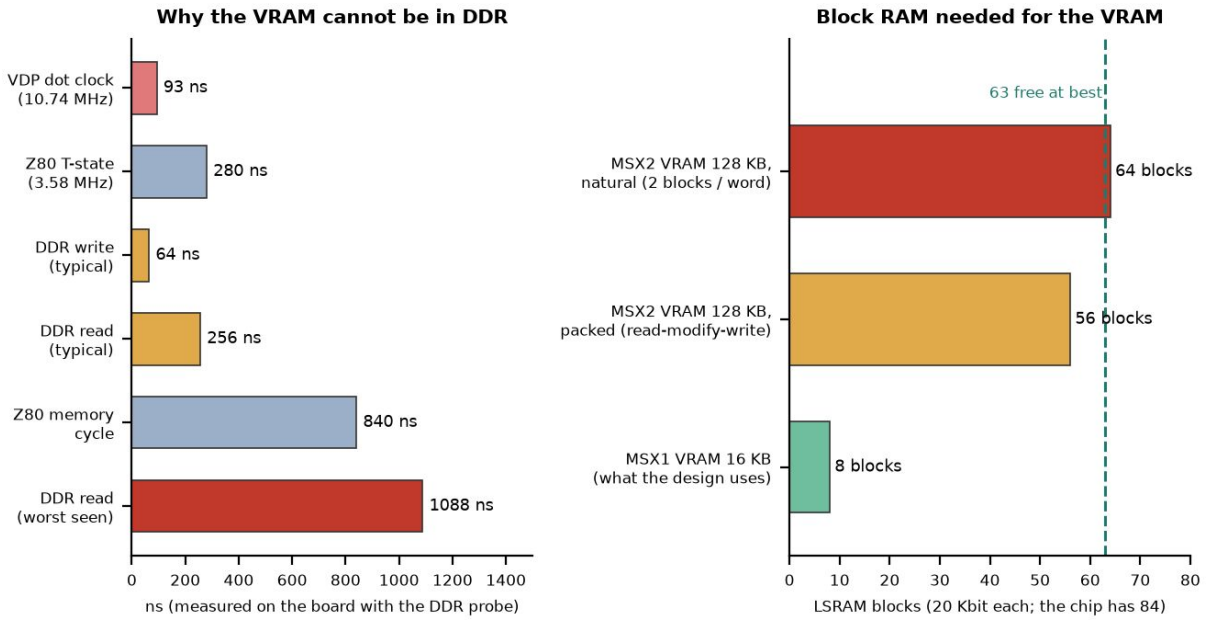


Figure 11. Left: the time scales involved (the VDP dot clock and the Z80's cycle against the measured DDR latencies). Right: the block RAM needed for a VRAM, against the 63 blocks that the pool can free at best.

A Z80 memory cycle at 3.58 MHz is about 840 ns, so a 256 ns read started when the address is valid fits inside one cycle, and the rare 1 µs outlier is one wait state (`WAIT_n`). The CPU can use far memory. A video chip is the opposite. The TMS9918A (and the V9938, and the MC6847) fetch their memory on a fixed schedule of a few dot clocks, one dot being 93 ns at 10.74 MHz, and have no wait input: a late byte is a wrong pixel. So the rule that

came out of the probe, and that every machine since has followed, is **CPU-side memory may wait; video memory may not.**

9.2 Where each memory of the MSX1 lives

Table 10. The MSX1's memories.

Memory	Size	Where	Why
Main RAM (slot 3)	64 KB	the pool (block RAM)	no wait states; shared with the other machines' RAM, so it costs nothing extra
VRAM	**16 KB**	**the pool: eight blocks** (the CP-400's ROM blocks)	the VDP cannot wait; 16 KB is small
BIOS, cartridge, disk ROM	up to 96 KB	DDR + 8 KB read cache	read-only, sequential instruction fetch (the cache hits), the Z80 can wait on a miss
Frame buffer	24 KB	the pool	shared with every machine's picture

9.3 The MSX2: what went wrong with 128 KB

The first MSX was an MSX2 (KdL's OneChip MSX with the V9958), because an MSX2 runs MSX1 software. Its VRAM is 128 KB, and that one memory broke the plan. In the original design everything (RAM, ROMs, VRAM) sits in one SDRAM behind a fixed-timing controller; the fabric has no SDRAM, only the DDR whose numbers are above. RAM and ROMs could move to DDR with wait states; the VRAM could not. The arithmetic of keeping it on the chip, with block RAMs of 20 Kbit (2K × 10) and 84 on the chip:

Table 11. The MSX2's VRAM options.

Way to keep a 128 KB VRAM	Blocks	Verdict
Natural mapping: 16-bit word = two blocks of 8 used bits	64	One block more than the pool can free (63 at best): not buildable at ~100 % of the chip.
Packed: five 16-bit words in eight blocks (row = adr[10:0], slot = adr[15:11]); a byte write is a read-modify-write of an 80-bit row	56	Fits, verified in simulation against a plain model (0 of 1.28 M ticks differ once writing stops). Costs ~2K LUTs of block-RAM interface logic and needs a two-tick write.
64 KB VRAM only (what the MSX2 standard allows)	32	No screens 6 - 8 and many games need 128 KB.
VRAM in DDR with a small cache	2 - 3	Tried as a design (write-through, ~4 KB in 64-byte lines); the VDP stalls on a miss for ~0.45 µs. Estimated ~1 % stall for display scanning but several times slower VDP blitter commands. With it the three-machine design still reached 24.07K LUTs. Abandoned.

The MSX2 branch built and simulated (C-BIOS logo and text screen, screen 5 and 7 byte ramps bit-exact, the packed VRAM identical to a plain one) at 20.0K LUTs and 76 block RAMs, but together with the CP-400 it needed 24.7K LUTs and 87 block RAMs on a chip with 22,956 and 84. The choice (2026-11-01) was to drop the MSX2 and build an MSX1 with a ZX Spectrum and an MC-1000 instead: all of them smaller, sharing one Z80, with VRAMs of 16 KB or less.

9.4 Is it still true?

Yes, in a smaller form. The MSX1's VRAM is 16 KB, **eight pool blocks, on the chip**, for the same reason as before: the VDP's fetch schedule. What changed is that the amount is small enough that the question stopped costing anything: the blocks are the CP-400's ROM blocks reused, so the VRAM is free. The question will come back for any machine with 64 KB or more of video memory (an MSX2, a Mega Drive, a SAM Coupé): that

machine needs either a smaller chip share for its other memories, a bigger chip, or a cache design whose cost has to be re-estimated with its blitter workload in mind. We have the numbers; we do not have a free 56 blocks.

10. Disks, tape and sound

10.1 One WD1793, three front ends

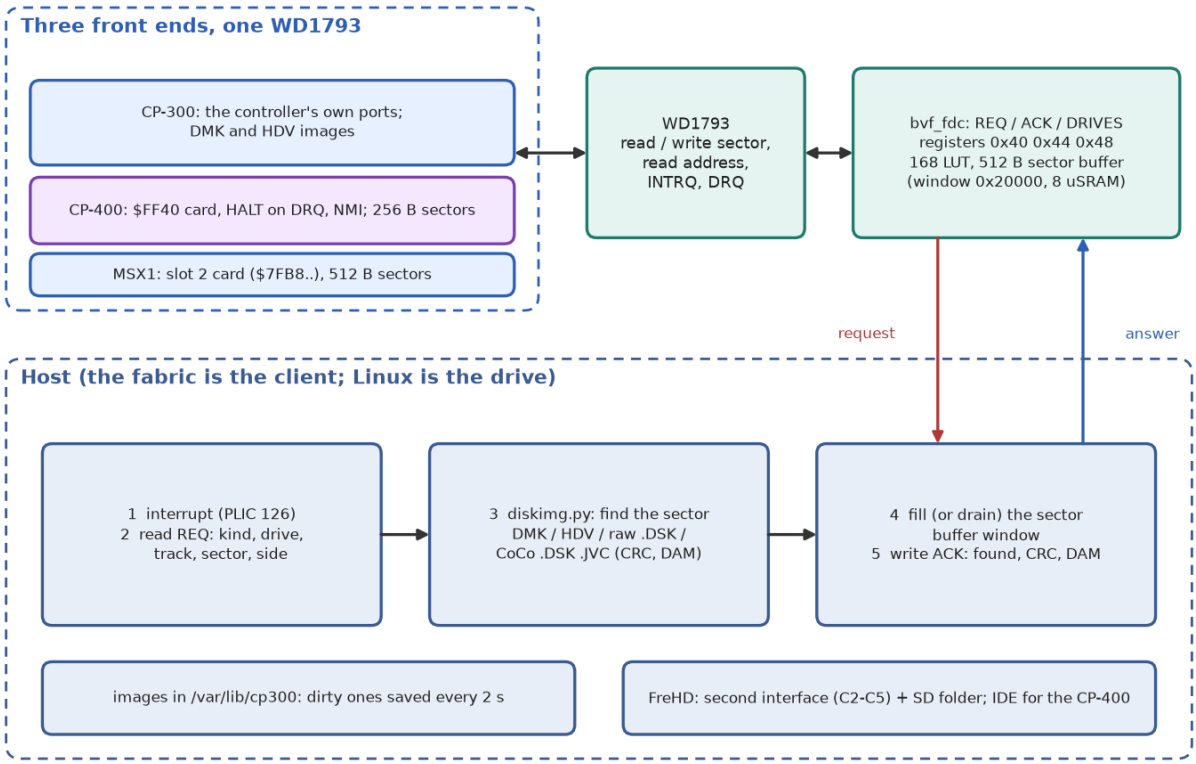


Figure 12. The sector service. Three machines have a floppy controller front end, all on the one WD1793 (a second bus on the CP-300 core); every sector request goes to the daemon, which finds the sector in an image file.

Table 12. The floppy front ends.

Front end	Where	Sector	Flow control	Images
CP-300	the TRS-80 Model III's controller ports	up to 512 (DMK)	the controller's own	DMK (JV1, JV3, HFE converted by `dsk2dmk.py`), HDV for the WD1010
CP-400	`$FF40-$FF4F` (`bvf_cocofdc`)	256, `sec512 = 0`	HALT = halt-enable and not DRQ; INTRQ clears the enable; NMI = INTRQ and density	CoCo `.DSK`, `.JVC` (35 × 18 × 256 = 161,280 bytes)
MSX1	slot 2, `$7FB8-$7FBC`	512, `sec512 = 1`	the ROM polls `$7FBC`: b7 INTRQ, b6 /DRQ	raw `.DSK` (720 KB, 360 KB, 640 KB, 320 KB)

10.2 The request/response protocol

`bvf_fdc` (168 LUTs, 177 flip-flops) turns a controller's sector command into a request the host can read: `REQ` (read sector, write sector, read address; side; drive; track; sector; hard-disk bit), a 512-byte buffer in the sector window (its shadow RAM is eight uSRAMs, no block RAM) and `ACK` (b0 = the buffer belongs to the host; found, CRC error, deleted-mark bits). `DRIVES` carries the ready and write-protect flags of four floppies and two hard disks and **keeps its value while the core is held**, so the daemon sets the flags before releasing the machine. The daemon (`diskimg.py`, a Python port of the Pocket firmware's C code plus the HDV, raw and CoCo formats)

answers one sector per request: a pending request raises the interrupt, the daemon reads `REQ`, fills or drains the window and writes `ACK`; the controller waits, so a slow answer slows the disk but never corrupts it. Writes land in the daemon's in-memory copy and are saved to `/var/lib/cp300` every two seconds if the bytes actually changed.

Latency. With one request per sector the interrupt-to-answer time becomes the speed of the disk. The counters in the daemon's statistics (`lat_max_ms`, `lat_slow`, `lat_last_slow`) showed about **35 ms per request**: a CP-300 `DIR` of about 1,700 sectors took over a minute. The likely cause was Python's thread switching and the picture work competing with the disk service; the fix in the daemon (a 0.5 ms switch interval, the picture limited to 10 frames per second while a disk request has just been answered, a stale interrupt timestamp removed) was tested against simulated registers and is reported as not yet confirmed on the board at the time of writing.

10.3 Building a front end for a ROM you cannot change: the MSX disk interface

The MSX1's disk interface is the Gradiente Expert DD Plus's. We have its 16 KB disk ROM but no schematic, so the controller was modelled by disassembling the ROM's own accesses. It worked in simulation immediately and then took **three corrections on real software**, each found only because the real ROM runs against it:

Table 13. The MSX disk interface: corrections in the order found.

Symptom on the board	Cause	Fix	How it was found
"Illegal function call in 0" instead of Disk BASIC	the disk ROM had been loaded into slot 0 page 2, so the BIOS ran it as a BASIC program	a separate slot for the disk ROM (slot 2, page 1), an eject button for the page-2 ROM	the message itself
"Disk offline"	`$7FBC` bits 1:0 select the drive **one-hot** (1 = A, 2 = B); the gateware decoded them as binary	one-hot decode (gateware 4325)	disassembling the ROM's writes: the ROM writes `$09` for drive A (motor bit + one-hot drive bit)
"Disk I/O error"	bit 6 of the `$7FBC` read is **/DRQ**: the ROM's transfer loop runs while it is 0; the gateware had it active-high	invert it (4326)	the transfer loop's `BIT 6,A` / branch, and the simulation's own test program had the same wrong assumption
Black screen after the splash, only with the disk interface present	the ROM copies its sector routine into RAM near `$0000` and runs it with the card selected in **page 2**: it reads `$BFB8`, `$BFBB`, `$BFBC`; the decode answered only in page 1, so the loop polled an empty bus forever	the registers also answer in page 2; the ROM does not (4327)	the daemon's Z80 bus sampler showed the CPU looping in low RAM and reading `BFB8`; the ROM's code has `LD BC,BFBC` / `LD (BFB8),A`

After the third correction (package 0.14.4) the MSX1 boots MSX-DOS from the floppy image on the board. *The test program in the simulation had been written from our understanding of the interface, so it passed all three times.* A bench that models the device from the same belief as the device cannot find a wrong belief; the real driver is the only test that can (L9).

10.4 The CP-400's hard disk: two puzzles

The MiniIDE passed its simulation with the real HDB-DOS cartridge, then said "LBA HARD DRIVE NOT FOUND" on the board. Two causes, found with a register poller on the board and, for the second, by running the DOS's steps by hand from CP-400 BASIC (`POKE`/`PEEK` of `$FF50-$FF58` is a good bus probe):

- **Host latency.** The daemon's single loop was busy with the 24 KB picture when the IDENTIFY request arrived and answered after 166 ms; the DOS gives up after a short wait. *Fix:* the pump serves disk requests only, and the picture, audio and saving moved to a lower-priority path with the XOR and zlib in threads.
- **A bit that never shows.** The IDENTIFY request was encoded as `3'b100`. On the board the request kind read as 0: bit 2 never appeared on the bus, although the simulation of the same RTL (also over the register bus)

shows 4 and the synthesised netlist has the flop. The cause was not found. *Work-around:* encode IDENTIFY as 3 (read and write: no command is both) so the next gateware does not use bit 2, and let the daemon treat kinds 0, 3 and 4 as IDENTIFY.

- **Word order of the data port.** The DOS found the drive but the file names were scrambled. The simulation, run with the DOS on the user's real sectors, printed exactly the screen of the board, so *the simulation was the oracle*: iterating in simulation (no board) found the rule (the DOS stores a pair [$FF50 of iteration j+1, $FF58 of iteration j]) and the window order that inverts it. The user's 507 MB OS-9 image then booted. Sector writes are still refused: the last byte pair cannot be recovered from the lagging low bytes.

10.5 Tape and sound

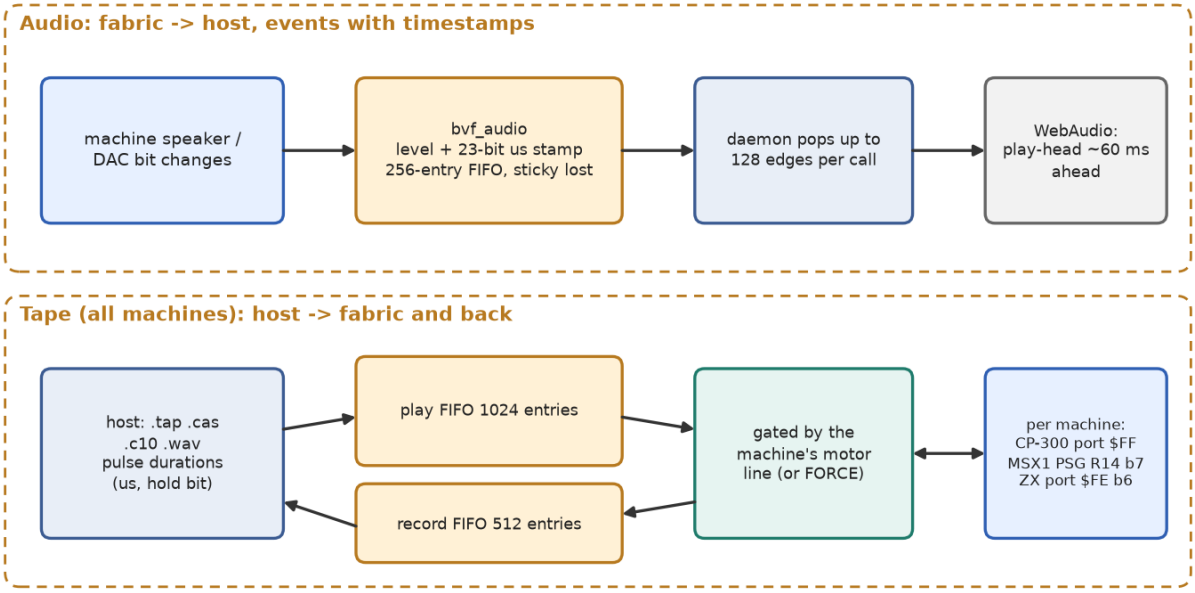


Figure 13. Sound goes from the fabric to the host as timestamped edges; the cassette recorder (all machines) is fed by the host and gated by the machine's motor line.

Sound (decision D6): `bvf_audio` compares the machine's speaker value (0 to 511) with the previous one every clock and pushes `{level[8:0], time[22:0]}`; time is a free-running microsecond counter (wraps at 8.39 s). A 256-entry FIFO sits between fabric and host; on overflow new edges are dropped and a sticky `lost` flag is set. The browser converts timestamp differences to sample counts and schedules them about 60 ms ahead of the audio clock, so latency is a constant, not a distortion.

Tape (`bvf_tape`, shared by all machines): the host pushes pulse durations (microseconds, with a hold bit for pauses over 2^{20} µs) into a 1,024-entry FIFO; the input level toggles at the end of each entry; time runs only while PLAY is on and the machine's motor is on (or FORCE), like a recorder whose motor the computer switches. The record side stores every change of the machine's output as `{level, microseconds since the previous change}` in a 512-entry FIFO. Per machine the connection is: CP-300 port `$FF`; CP-400 PIA1 PA0 / the DAC's top bit / CA2 motor; MSX1 PSG register 14 bit 7 / PPI C bits 5 and 4; ZX port `$FE` bits 6 and 3; MC-1000 PSG ports B7 / A7; the Alice inside the emulator (the same entries over its stdin). Formats: `.tap`, MSX `.cas` (1200 baud), CoCo/MC-10 `.cas` / `.c10` (1200/2400 Hz), TRS-80 `.cas` (500 baud), `.wav`; recording gives a `.wav`. Simulated; the header lengths and polarities the ROMs expect are from the formats' documents and have not been tried with real programs on a board.

11. The picture path

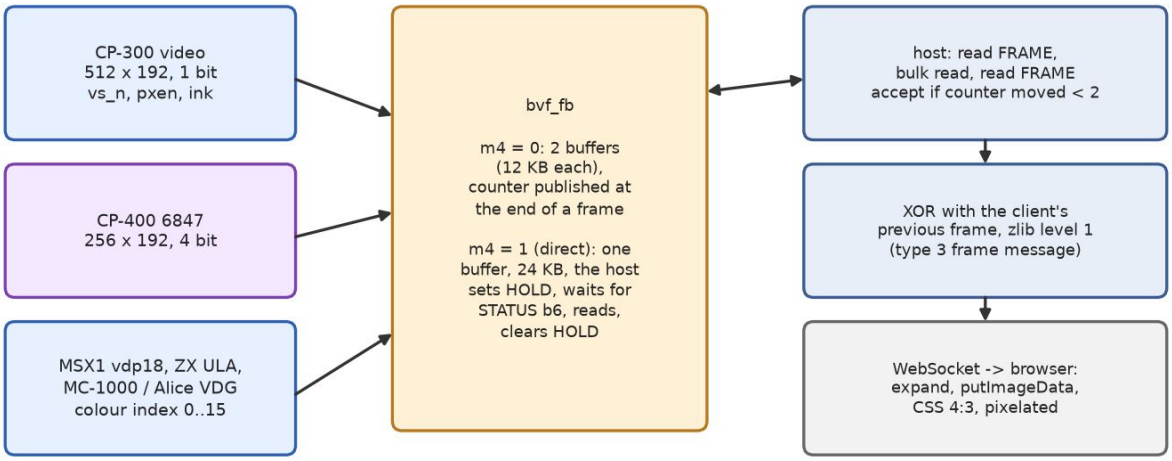


Figure 14. Every machine's picture goes through `bvf_fb`, in one of two modes, and the host reads it in bulk.

11.1 Capture

The machines already produce a pixel stream with a pixel enable, blanking signals and a pixel value. `bvf_fb` counts x and y, shifts pixels into 32-bit words and writes them to RAM. Two modes:

- **CP-300 (m4 = 0):** 512 × 192 × 1 bit = 12,288 bytes = 3,072 words, `word = y × 16 + x / 32`, pixel x in bit (x mod 32), leftmost pixel least significant, so a little-endian host sees LSB-first bytes in reading order and never shuffles bits. Two buffers; the writer publishes (`latest`, a 16-bit counter) when a frame ends and flips.
- **Everything else (m4 = 1, "direct"):** 256 × 192 × 4 bit colour index (0 to 15; the page owns the palette) = 24,576 bytes, one buffer. The host sets HOLD (`CTRL` b6), waits for `STATUS` b6 (no frame is being written), reads the words and clears HOLD; a new frame is not started while HOLD is set.

11.2 Tear-free reads

The CP-300's capture must never wait for Linux, so it is lock-free: the host reads the counter, bulk-reads the newest buffer, reads the counter again, and accepts the copy only if the counter advanced by less than two. A copy taken while the writer lapped the reader is discarded and retried (the daemon counts `stats.dropped`). The generalisation holds for any rate-limited stream the host samples: *double buffer plus a monotonic counter read before and after*. The colour machines can afford HOLD because their picture is regenerated every frame from machine state that is stable while the picture is skipped.

11.3 To the browser

Each client receives the XOR of the current frame with its own previous frame, compressed with zlib level 1; most pixels do not change, so a frame becomes a few hundred bytes. A new client gets a whole frame (kind 0). The page applies an XOR frame only if it has the previous frame and the sequence number follows; otherwise it drops its copy and asks for a key frame, which makes the daemon clear that client's previous frame. The page

expands the bytes to RGBA, draws them with `putImageData` and scales the canvas by CSS to a **4:3** box with `image-rendering: pixelated`: the real tube displays 512 × 192 on a 4:3 screen, so each dot is twice as wide and four times as tall. *The fidelity of a retro display is a property of its geometry, not of the framebuffer.*

A one-pixel trap. The MSX1's picture was clipped by one pixel at the left edge: `bvf_fb` takes the colour one clock after the pixel enable but the blanking from the enable edge itself, so the VDP's colour needed one more clock of delay (`col_q` in `bvf_msx1.v`). A testbench of the capture alone (`tb_vdp18win`) now guards it.

Speed. Measured in the page: the CP-300 about 38 - 40 frames per second against a 60 Hz source, the CP-400 about 12 (package 0.6.0, before the disk-latency work). The bus is never the bottleneck (a bulk read is 1.5 ms for 12 KB); the Python XOR over the frame and the WebSocket framing are. Moving the XOR into the helper library, or into the fabric, would remove it.

12. The host side: daemon, protocol and page

12.1 Register access: regs32.c

Ten lines of C, compiled on first use with `gcc` and called through `ctypes`, issue real 32-bit loads and stores; a block routine reads a whole frame in one call (a 6,144-word read through `ctypes` word by word would spend its time in call overhead):

```
uint32_t rd32(uintptr_t a)             { return *(volatile uint32_t *)a; }
void     wr32(uintptr_t a, uint32_t v) { *(volatile uint32_t *)a = v; }
void rdblk(uintptr_t a, uint32_t *dst, size_t words);   // word loop, volatile
void wrblk(uintptr_t a, const uint32_t *src, size_t words);
```

12.2 The daemon and its simulator

`host/cp300d.py` is standard-library Python only (no pip packages, so the package installs on the stock image). The `Cp300` object mirrors the interface contract: `start()` (the power-on sequence of section 6.3), `set_machine()`, `mount()` / `eject()` for each disk class, `service()` for the sector mailbox, the FreHD and IDE services, `new_frame()`, `audio_events()` and the tape deck. A `SimRegs` class implements the same register interface against a Python model, so **the daemon and the page can be developed and tested with no hardware** (`--sim`; `test_cp300d.py` is a test suite built on it). Writing the register file as a replaceable class was probably the best productivity decision of the project: the whole web side was finished before the gateware was.

12.3 The pump

One `asyncio` task is the hardware loop. Priority order: **anything the hardware blocks on is served before anything the user sees.** Disk, FreHD and IDE requests first (interrupt-driven, 0.5 to 2 ms fallback polling); then the picture (at most once per hardware frame; for each client the XOR of the frame with its previous frame, zlib level 1); then audio edges; every two seconds dirty disk images are saved. The picture, audio and saving run in a lower-priority path with the register reads, XOR (as big integers) and compression in threads, because the IDE lesson (section 10.4) showed that one slow picture can make a DOS give up on its disk. When the last browser disconnects the keyboard registers are zeroed, so no key stays pressed.

12.4 A bring-up probe built into the product

`GET /api/debug` returns the control, status, frame, audio-count and tape registers, the time of 1,000 register reads and of one frame read, a **write-readback test** (an `0xA5A55A5A` pattern and an eight-byte block over the keys registers, to detect byte-store corruption), the interrupt count, the frame rate, and, since the multi-machine work, **the selected machine's internal state**: the CP-400's 6809 address and floppy card lines, or the shared Z80's bus

sampled 32 times (`z80_bus`: address and MREQ/IORQ/RD/WR/M1 flags, so *the address that repeats is where the CPU is*), and for the Spectrum the ULA's last fetches. The probe writes nothing that disturbs a running machine, and waits with `await` rather than `sleep` so the daemon keeps serving disk requests. It found four of this project's bugs (L1 to L3, and the MSX disk hang). If you build one thing before your first board bring-up, build this.

12.5 The web protocol

One HTTP server on `asyncio.start_server` with a hand-written WebSocket (about forty lines: handshake, frame encode/decode with client masking, ping/pong). No third-party dependency; no TLS (the board's nginx or Cockpit can terminate it); request bodies above 4 MB get HTTP 413, matching nginx.

Table 14. The main endpoints.

Endpoint	Use
`GET /`, `GET /ws`	the page; the WebSocket
`GET /api/state`, `GET /api/debug`	JSON state (machines, ROM presence, drives, SD files, tape, counters); the probe of section 12.4
`POST /api/ctrl`	`{machine, power, reset, turbo, wp}`: select a machine, the power switch, reset, turbo, write-protect
`PUT /api/rom/<name>`	`system`, `chargen`, `zx48`, `mc1000`, `alice`, `msx1bios`, `msx1p2`, `msx1cart`, `msx1disk`, `cp400bas`, `cp400ext`, `cp400cart`, or `cp400` (a complete image); saved, and the machine is (re)started when its ROMs are complete; wrong sizes get HTTP 400
`PUT/GET/DELETE /api/disk/N`, `/api/cocodisk/N`, `/api/msxdisk/N`	mount (converted to DMK for the CP-300), download, eject
`PUT/GET/DELETE /api/ide`, `/api/hd/N`, `/api/sd`	the CP-400's hard disk image; the CP-300's hard disks; the FreHD SD folder
`PUT/DELETE/POST /api/tape`, `GET /api/tape/rec.wav`	load a tape in any supported format (converted in the daemon), eject, transport commands; fetch the recording as a `.wav`

WebSocket messages: binary frames carry `type | seq u16 | kind u8 | zlib(payload)` (type 1: 12 KB one-bit frames; type 3: 24 KB colour frames; kind 0 = whole frame, kind 1 = XOR with the previous one) and audio edges `(time u32, level u16) × n`; text messages carry the JSON state (pushed on connect and on every change). The browser sends `{keys: <hex>}` (the **whole** key state on every change: a lost or reordered message cannot leave a key stuck, and the daemon does no key logic), `{ctrl: ...}` and `{keyframe: 1}`.

12.6 The page

One self-contained file, no framework, no build step. Keyboard mapping is from *characters* to the machine's *matrix*, not from physical keys, with a table of how each original keyboard types its shifted symbols; the Alice has its AZERTY variants; an on-screen keyboard from the same table serves a tablet. Audio needs a user gesture, so the Sound button creates the `AudioContext`. URLs are relative, so the page works under `/cp300/` behind nginx. Drive names are inserted with `textContent`. The page title and the Cockpit entry are both "Retro Computers".

13. Delivery: device tree, build, package, installer

13.1 The device-tree overlay

```
&{/chosen} { overlays { CP300-GATEWARE = "GATEWARE_GIT_VERSION"; }; };      // the identity the OS can read
```

```
&{/fabric-bus@40000000} {
    cp300: cp300@41000000 { compatible = "generic-uio"; reg = <0x0 0x41000000 0x0 0x100000>;
                            clocks = <&fabric_clk3>; interrupt-parent = <&plic>; interrupts = <126>;
                            status = "okay"; };
};
```

- `/chosen/overlays/CP300-GATEWARE` is how the board reports which gateware is loaded. The installer and the Cockpit page read it to decide "is this design loaded or did someone flash another?".
- `generic-uio` makes `/dev/uio0` with name `cp300`; the systemd unit refuses to start the daemon unless `/sys/class/uio/uio0/name` reads `cp300`, so a different gateware can never be driven by this software. The service runs as an ordinary user; a udev rule grants the device.
- `interrupts = <126>`: fabric interrupts are PLIC 118 + n, and the cape's `INT[0]` is `MSS_INT_F2M[8]`. The number was not in any table we had; the board's own working devices (PCIe root port on F2M[1] = PLIC 119, the IHC mailbox on F2M[62] = 180) were taken as the reference.

13.2 Building the gateware

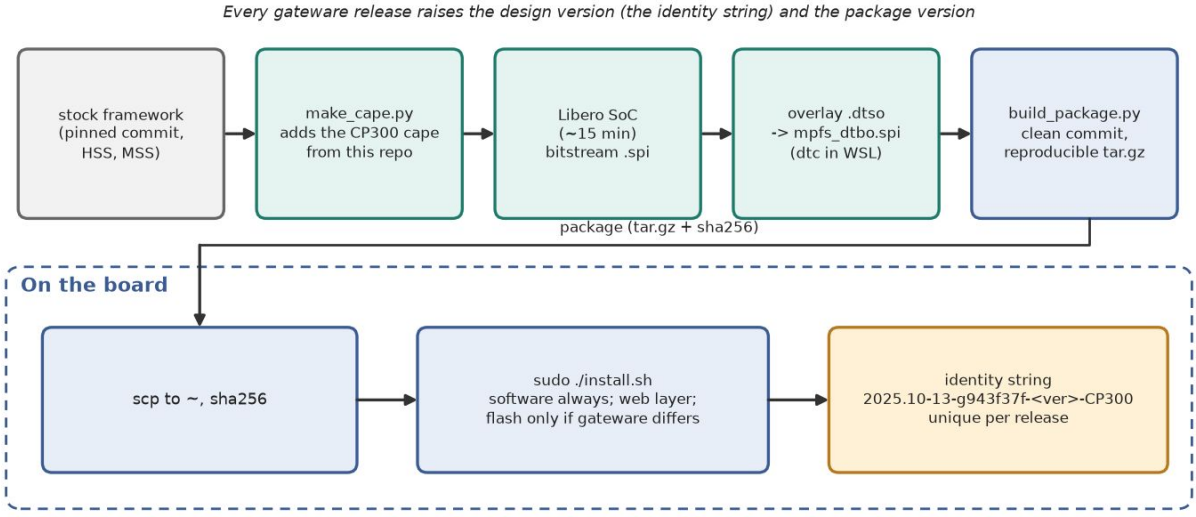


Figure 15. From the repository to a running board. The identity string and the package version are raised for every gateware release.

1. Clone the board's gateware framework at the commit of its stock gateware (`943f37f`, 2025.10-13) and generate the MSS configuration with `pfsoc_mss`; build the HSS at the commit the framework pins.
2. `python gateware/make_cape.py <framework>` adds `CAPE_OPTION:CP300`: it copies the template cape, replaces its register block with this design and a PLL, adds `PREADY`/`PSLVERR` and the AXI ports to the cape, routes the interrupt, stages the HDL (stripping `translate_off` blocks that Libero's parser ignores) and writes the import script and the overlay. **Every edit of the template must match exactly once or the script stops** (L5).
3. Run the framework's build in Libero SoC 2026.1 with a new `DESIGN_VERSION` (about 13 to 15 minutes; the log is written only at the end). The framework needs `AUTO_COMPILE_POINT:false` before synthesis: with automatic compile points Synplify crashed loading a compile point's interface model. The Synplify licence is node-locked, so **one build at a time per machine**.
4. Build the overlay image `mpfs_dtbo.spi` (the framework's overlay generator with `dtc`, in WSL), giving it the identity `2025.10-13-g943f37f-<DESIGN_VERSION>-CP300`.
5. `package/build_package.py --gateware <dir> --version X.Y.Z` from a **clean committed tree**: a reproducible `tar.gz` (sorted entries, fixed owner and time) and a SHA-256.

Two rules carry most of the weight. **Pin the framework to the stock gateware's version**: a framework newer or older than the bootloader and MSS the board shipped with can give a design that boots to nothing (the `develop`

branch turned out to be from 2023, older than the board's gateware). **The identity string must be unique per design *and per release***, and the design version must differ from what is loaded, or the board will not reprogram; the installer flashes only when the loaded identity differs (an early package used an identity without the design version, so boards never got new gateware).

13.3 The package and the installer

The package holds the installer, the load helper, the daemon, the page, `dsk2dmk.py`, the service, nginx and Cockpit files, the two flash images, `VERSION`, `GATEWARE_ID`, `SOURCE.txt` (source URL, commit, checksums), the README, the licences and a disclaimer. The ROMs are never in it. `install.sh` is idempotent and layered by risk:

Table 15. The installer's layers.

Layer	Risk	Behaviour
Software (`/opt/cp300`, service, udev rule)	none	always replaced; `regs32.so` and the Alice emulator are compiled on the board at install
Web integration (nginx snippet, Cockpit entry)	low	replaced; the stock site file is backed up once, outside `sites-enabled`; one `include` line is added and `--uninstall` removes it
Gateware flash (bitstream, overlay, HSS)	**can brick the board**	only if the loaded identity differs; shows the disclaimer; requires typing `I ACCEPT` (or `--yes`)

Options `--yes`, `--no-flash`, `--no-web`, `--force-flash`, `--check` and `--uninstall [--purge]` make each layer reachable on its own; `--check` changes nothing and reports the installed version, the loaded gateware and the daemon state: the first command a support conversation should ask for. The service starts only if this gateware is the loaded one: flashing a different design leaves the software dormant, and `cp300-load-gateware` brings the design back. Cockpit gets a sidebar page that opens the web page in a new tab (an https Cockpit cannot frame an http page, and its content-security policy forbids inline scripts).

14. Performance and resources

Table 16. Measured numbers.

Quantity	Value	Note
Machine clock	42 MHz	PLL from the 50 MHz PCLK; machines use clock enables (CP-400 15/44, MSX1 45/88, ZX 1/12)
Logic (gateware 4328)	**21,659 of 22,956 4LUT (94.35 %)**	the margin is 1.3K LUTs; timing met on every build (worst slack of the 42 MHz domain +9.7 ns with the CP-300 alone, +4.9 ns later with DDR)
Block RAM	80 of 84 LSRAM (gateware 4322)	56 in the pool; the rest in the CP-300 core, the IDE sector buffer and others
Largest blocks	`cpu09` 2,942 LUT; T80s 2.3-2.6K; the pool's muxes ~1.6K	block-RAM interface logic is ~36 LUTs per block
Register window	1 MB used of 16 MB	`0x41000000`; FIC0 AXI for DDR, 125 MHz
Bulk frame read	about 1.5 ms (12 KB)	one `rdblk` call
Page frame rate	CP-300 38-40 fps; CP-400 about 12 fps	Python XOR and framing are the limit
Disk request	about 35 ms interrupt-to-answer before the latency work	target a few ms; not yet confirmed on the board
DDR read, write	256 ns, 64 ns typical; 1.09 µs, 520 ns worst	measured with the probe

Quantity	Value	Note
Build	13 to 15 minutes in Libero	on the author's PC

Where the time goes, in the order we would fix it: (1) the Python XOR over the frame (do it in C or in the fabric); (2) the 4 ms timer for the picture (a frame interrupt would wake the loop once per frame); (3) the disk path's thread scheduling; (4) the area: the DDR probe is about 1.2K LUTs and is only needed for measuring.

15. Lessons learned

L1 to L8 are from the first port and still hold; L9 to L16 came with the other machines.

L1. The window has no byte strobes

Symptom: a written word read back as one byte repeated in all four lanes; data through the window was corrupt. *Cause:* the window is an APB3 bus with no `PSTRB`; a CPU byte store sends the whole data bus, and Python's `memcpy` and `struct.pack_into` perform byte stores on this core. *Rule:* prove 32-bit read/write integrity with a probe before anything else, and make "never a byte access" a property of one small module.

L2. Reset order across a held state

The machine never started although every register looked right: the slot-table registers are cleared while `run = 0`, and the daemon had published them first and then released `run`. *Rule:* write down which registers belong to which reset domain and make the init sequence follow it (section 6.3).

L3. Torn frames

A reader of a fabric-written buffer needs a consistency check designed in, not retrofitted (section 11.2).

L4. The toolchain version is part of the design

Framework, MSS configuration, bootloader and synthesis tool must be a matching set. Identify the exact stock build first, match it, and treat every deviation (here: an unofficial Libero version needing two scripted edits) as a known risk in the disclaimer.

L5. A patch step that can find nothing must be an error

`make_cape.py` connected the interrupt with an exact-text replace that matched nothing, and a package shipped with the interrupt tied to zero; nobody noticed because nothing used it yet. Every edit of a vendor template must match exactly once or stop the script. Likewise a small array with a reset was inferred as RAM and rejected by Libero (`MF682`), and `translate_off` pragmas were ignored: script the adaptations so that they are reproducible.

L6. Systems integration is half the work

None of these were gateware bugs: a duplicate default server in nginx because a backup file had been left in `sites-enabled`; the Cockpit page stuck at "checking" because of its content-security policy; an https page that cannot frame an http one; a path prefix that broke absolute URLs; line endings carried from Windows. Budget as much for the host's stock software as for the core.

L7. Build the whole host side against a simulator first

`SimRegs` made the daemon and the page testable off-board; when hardware arrived the only failures were interface ones, which a probe like `/api/debug` isolates.

L8. Level semantics beat event semantics

Keys as full masks, `changed` as the XOR of two registers, a request as a toggle pair: each removes a class of lost-event bugs at no cost.

L9. A bench built from the same belief as the design proves nothing about the belief

The MSX disk interface passed its simulation and failed three times on the board (section 10.3): the test program had been written from the same understanding of the interface as the gateware. The only test that can find a wrong belief is the real driver. *Rule:* for any device with a driver you did not write, disassemble the driver first, and make the real ROM the test.

L10. A design that works or not depending on the build

The ZX Spectrum picture is black on gateware 4326 and 4327 and correct on 4325 and 4328; the Spectrum logic is identical in all of them and the simulation always shows the picture. The only changed RTL was in the MSX1. Timing was met on every build, so the timing report is not flagging it. We could not isolate the cause in the time available; it looks like a hardware dependency of the single-read-port arrangement that simulation cannot see (read-during-write behaviour of the block RAMs, or a path that is not what the constraints say). *What we can say:* put observability in before you need it (the debug register and the bus sampler found the MSX hang in one look; the Spectrum's ULA word showed the symptom but not yet the cause), and **treat "timing met" as necessary, not sufficient, for a design that shares one memory port between two clients.**

L11. The simulation is an oracle only if it runs the real software

The CP-400's hard disk could be debugged without the board once the simulation ran the DOS on the user's real sectors and printed exactly the board's screen. Then iterating on the word-order rule took minutes, not flash cycles.

L12. A bit that never shows

A register bit (bit 2 of the IDE request kind) never appeared on the board while it did in simulation and in the netlist. Do not spend a day on it: encode around it, make the host tolerant, write it down, and move on.

L13. Area is the real specification

The MSX2 was a fine design that could not exist next to the other machines. Every machine added after it was chosen by what fits: the 94 % LUT figure is not an accident but the result of choosing smaller machines (MSX1 over MSX2) and moving work off the chip (disk firmware, the Alice's CPU). Keep a running table of LUTs and block RAMs per block (Figure 3 and section 14), and refuse a feature whose cost you have not measured.

L14. Measure the memory before you put a machine in it

The DDR probe cost one gateware build and decided the memory architecture of three machines in one afternoon of measurements. Without it we would have argued from datasheets.

L15. Share the CPU, but write down its bus-cycle rules

A shared T80s works because every machine accepts the same cycle shape: writes at the end of /WR, `$FF` on the interrupt acknowledge, data taken at the enable. Each of those rules was found as a bug in one machine and then applied to all.

L16. The build farm is one licence

The synthesis licence is node-locked: one build at a time on the machine, so parallel sessions must serialise their builds and keep their own framework copies (their run scripts delete the working folder first). Keep every run script, overlay script and log of every gateware you flash.

16. What is verified and what is not

State on 2 November 2026, gateware 4328, package 0.14.5. **Board** = seen running on the author's BeagleV-Fire (Ubuntu 24.04 image of 2025-07-21, stock gateware 2025.10-13). **Sim** = ModelSim, Icarus or the Python tests on the PC.

Table 17. Verification status.

Machine / feature	Verified on the board	Simulated only	Never run / unknown
CP-300	boots TRSDOS and LDOS, games, keyboard, sound, FreHD boot disk + SD images, `DIR` on hard disks	-	the latency fix of section 10.2
CP-400	BASIC, MiniIDE + HDB-DOS (`DIR`), the 507 MB OS-9 image boots, floppy: Disk BASIC banner and 340 sector reads with a CoCo image	floppy `DIR`/`LOADM` end to end	IDE sector writes (refused), joysticks, serial
MSX1	Elevator Action cartridge, BASIC, **MSX-DOS boots from a floppy image (0.14.4)**	tape input	other disk-interface layouts, FORMAT, a cartridge larger than 32 KB
ZX Spectrum 48K	boots to the TK90X screen on gateware 4325 and 4328	the picture on every build	**black screen on 4326 and 4327 (L10)**; tape with real programs
MC-1000	BASIC prompt	boot to `OK` at 1.5 s	-
Alice (MC-10)	-	the emulator (93 checks, MC-10 ROM to the prompt), the fabric half (prompt, semigraphics, RG6)	the emulator on the board, sound jitter, field sync
Tape, all machines	-	unit bench; the MSX1's cassette input counted by a Z80	real tapes: header lengths and polarities
Installer, flash	the software path; the "already loaded, do not flash" path; the user's install and flash of each package	-	recovery with a FlashPro programmer; `--force-flash` and "another gateware, and back"

Flashing the board rewrites its bootloader with no fallback; the README and the package carry a disclaimer, and the installer asks for explicit acceptance.

17. A checklist for other projects

For any design that pairs an FPGA block with a CPU running an OS: a retro core, a sensor front end, a codec, an accelerator, a test instrument.

Interface

1. Choose **one memory-mapped register file** as the contract, plus **bulk windows** hanging off the same decoder, and version it with an ID register the software checks first.
2. Assume **32-bit, no-byte-strobe** access until proven otherwise; put all access in one tiny C helper.
3. Classify every flow: *state* (whole-value writes), *stream* (double buffer + counter, or a HOLD), *events* (timestamped FIFO), *request/response* (a mailbox with toggles). Do not force them into one mechanism.
4. Cross clock domains once, at the bridge, with a toggle handshake. Use level/idempotent semantics for what the host sets, counters for what it observes.
5. Document the **read latency** convention at the top of the register file. Give the host a **probe**, and add visibility (bus sampler, debug word) for every block that has a state you cannot otherwise see.

Architecture

1. Share what is never used at the same time (CPU, RAM, pools of block RAM, controllers) and write the rules of the shared interface down.
2. Move everything that is not hard real-time to the host, including, when the numbers say so, a whole CPU. Keep video generation, sound edges and bus cycles in the fabric.
3. Measure the slow memory before you move anything into it, and apply one rule: *CPU-side memory may wait, video memory may not.*
4. Keep a running table of area per block. The chip's size is the real requirement document.

Host software, platform and delivery

1. Develop the host against a **simulated register file** first; serve the hardware before the user; prefer a web UI for a headless device; keep format knowledge out of the RTL.
2. Identify the platform's stock build and match it exactly. Make the OS discover the design (device-tree overlay with a name and an identity), make the service conditional on it.
3. Ship a package and an installer layered by risk, with `--check` and an explicit acknowledgement before any irreversible step.
4. Test a modelled device with its **real driver**, not with a test written from your own understanding of it.

Appendix A. Register map

Byte offsets in the window (`rtl/bvf_regs.v`, ID `0x43503005`). The daemon (`host/cp300d.py`) and the gateware must carry the same ID.

Table 18. Registers.

Offset	Name	Contents
`00` r	ID	`0x43503005`
`04` rw	CTRL	b0 run (0 = hold), b1 reset pulse, b2 turbo, b3 write-protect all, b6 frame hold, b7 COLD pulse (power-on: clear RAM), b9:8 and b5:4 machine (0 CP-300, 1 CP-400, 2 MSX1, 3 ZX, 4 MC-1000, 5 Alice)
`08` r	STATUS	b1 FreHD command pending, b2 disk request pending, b3 request is a write, b5 audio FIFO lost, b6 frame buffer latest / no frame being written, b7 sector buffer copy running
`0C`, `10` rw	KEYS_LO, KEYS_HI	key matrix bits 31:0 and 63:32 (bit = 8 × row + column)
`14` r	FRAME	[15:0] frame counter, [16] buffer holding the newest frame
`18` rw, `1C` rw	AUDIO, AUDIO_HEAD	FIFO count; head `{level, time}`, a write pops it
`20` rw	KEYS2	[23:0] the MSX matrix rows 8..10 (bits 64..87)
`40` r	REQ	b0 read sector, b1 write sector, b2 read address, b3 side, b5:4 drive, b6 deleted mark, b7 hard-disk request, then head, track, sector
`44` rw	ACK	b0 the buffer belongs to the host; b8 found, b9 CRC error, b10 deleted mark
`48` rw	DRIVES	b3:0 floppy ready, b7:4 floppy write-protected, b9:8 hard disk ready, b11:10 hard disk protected (kept while run = 0)
`50`-`5C`	IDE_REQ, IDE_LBA, IDE_ACK, IDE_CFG	the CP-400's MiniIDE: request kind, LBA, completion + error, cartridge / disk / MSX1-interface present
`60`-`94`	DDR probe	CTRL (mode, start), ADDR, WDATA, COUNT, RDATA, LAT_MIN/MAX/SUM, TOTAL, RESULT, T42
`98`, `9C`	FRH_REQ, FRH_ACK	the FreHD command interface
`A0` rw	ALICE	[7:0] the `$BFFF` latch (b7 DAC); read: b8 = VRAM window busy
`A4`-`B0`	TAPE	control, play push, record count, record data

Offset	Name	Contents
B4 r	DEBUG	CP-400: 6809 address and floppy lines; other machines: the shared Z80's bus ([31:24] data, [20:16] M1/WR/RD/IORQ/MREQ, [15:0] address); ZX: ULA attribute and bitmap bits, last attribute write

Appendix B. Gateware history

Table 19. Gateware and package versions.

Gateware	Package	Content	4LUT
4302 / 4303	0.3.0 / 0.4.0	CP-300 + IO controller; the disk interrupt on the board	~19K (78 %)
4304	-	IO controller removed; host serves the sectors	9.2K
4307	0.6.0	+ CP-400	13.7K
4308	0.7.0	+ MiniIDE	14.5K
4310 / 4311	0.8.0 / 0.9.0	DDR probe; power cycle (COLD)	16.0K
4312	0.10.0	FreHD command interface	-
4313	0.11.0	CP-300 + MSX2 (V9958), own Z80, packed VRAM	20.0K (87 %)
4316	-	trial: shared Z80, MSX1 + ZX	19.5K (85 %)
4320 / 4321	0.12.0 / 0.12.1	six machines; MSX1 first-pixel fix	20.7K / 20.8K
4322	0.13.0	tape, MSX1 disk interface	21.4K (93 %)
4323	0.14.0	CP-400 floppy controller	21.6K
4324 - 4328	0.14.1 - 0.14.5	SAM page-1 fix + debug register; MSX disk drive select, /DRQ, page-2 decode; ZX debug word	21.66K (94.35 %)

Appendix C. Sources and licences

Table 20. Third-party parts. The combined bitstream includes the non-commercial PSG, so the project stays non-commercial (a decision of the owner, stated in the README and the package). No ROM image is ever committed or packaged.

Part	Origin	Licence
CP-300 core (cp300core/)	derived from the open HT1080Z core; the CP-300 FPGA project	GPL v3
T80 (third_party/T80)	mist-devel/T80	see the repository
CP-400 core (cp400/)	the author's CP400_COLOR_FPGA: Victor Trucco's Multicore core (based on Mark McDougall's PACE); cpu09 by John Kent	Trucco's files BSD-style; cpu09 GPL v3
MC-1000 VDG (mc1000/)	Victor Trucco's MC1000 core, renamed	BSD-style
TMS9918A (msx1/vdp18/)	Arnim Laeuger, from MSX1fpga	redistribution with the notices kept (the folder also carries GPL v2)
PSG (msx1/ocm/)	KdL's OCM-PLD, based on the ESE MSX-SYSTEM3 of ESE Artists' Factory	**not to be sold nor used in a commercial product or activity**; headers kept
MC6803 emulator (alice/emu)	written here, from MAME's mc10.cpp / m6801.cpp as the reference	GPL v3 with the project